EXPERIMENT 1A

ANALYZE THE TREND OF DATA SCIENCE JOB POSTINGS OVER THE LAST DECADE FOR THE GIVEN DATA ON THE NUMBER OF DATA SCIENCE JOB POSTINGS EACH YEAR. USE PANDAS FOR DATA MANIPULATION AND MATPLOTLIB / SEABORN FOR VISUALIZATION.

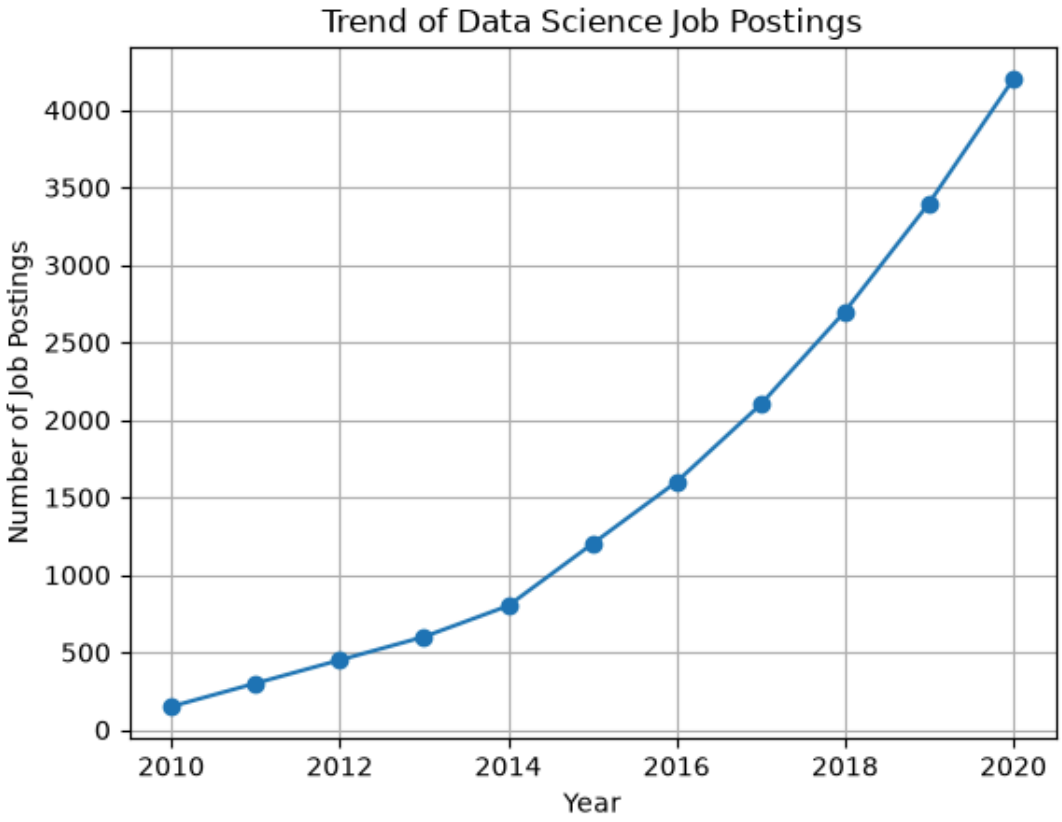
In [4]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [5]:
```
data = {
    "Year": list(range(2010, 2021)),
    "Job Postings": [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200],
}
df = pd.DataFrame(data)
print(df)
```

```
    Year  Job Postings
0   2010           150
1   2011           300
2   2012           450
3   2013           600
4   2014           800
5   2015          1200
6   2016          1600
7   2017          2100
8   2018          2700
9   2019          3400
10  2020          4200
```

In [6]:
```
plt.plot(df["Year"], df["Job Postings"], marker="o")
plt.title("Trend of Data Science Job Postings")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.grid(True)
plt.show()
```

OUTPUT


Trend of Data Science Job Postings
Number of Job Postings
0
500
1000
1500
2000
2500
3000
3500
4000
2010
2012
2014
2016
2018
2020
Year

EXPERIMENT 1B

ANALYZE AND VISUALIZE THE DISTRIBUTION OF VARIOUS DATA SCIENCE ROLES (DATA ANALYST, DATA ENGINEER, DATA SCIENTIST, ETC.) FROM A DATASET.

In [1]:
```
import matplotlib.pyplot as plt

roles = [
    "Data Analyst",
    "Data Engineer",
    "Data Scientist",
    "ML Engineer",
    "Business Analyst",
]
counts = [300, 500, 450, 200, 150]
```

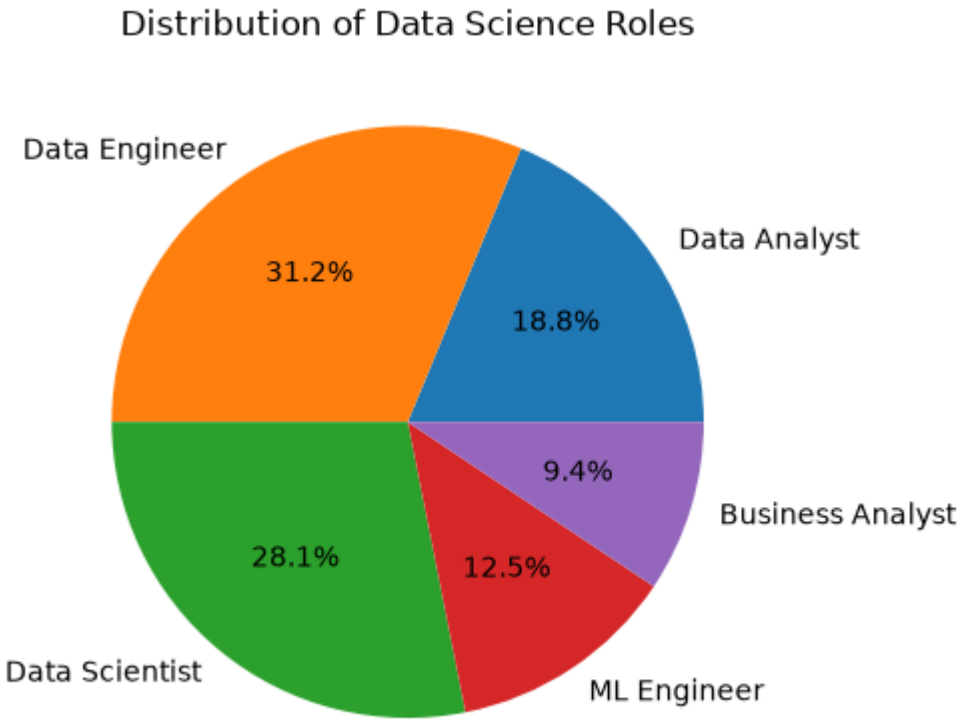
In [2]:
```
plt.bar(roles, counts)
plt.title("Distribution of Data Science Roles")
plt.xlabel("Role")
plt.ylabel("Count")
plt.xticks(rotation=15)
plt.show()
```

OUTPUT



In [3]:
```
plt.pie(counts, labels=roles, autopct="%1.1f%%")
plt.title("Distribution of Data Science Roles")
plt.show()
```

OUTPUT


Distribution of Data Science Roles
Data Engineer
31.2%
Data Analyst
18.8%
9.4%
Business Analyst
28.1%
12.5%
Data Scientist
ML Engineer

EXPERIMENT 1C

CONDUCT AN EXPERIMENT TO DIFFERENTIATE STRUCTURED, UN-STRUCTURED AND SEMI STRUCTURED DATA BASED ON DATA SETS GIVEN.

In [1]:
```
import pandas as pd
```

In [2]:
```
structured_data = pd.DataFrame(
    {"ID": [1, 2, 3], "Name": ["Alice", "Bob", "Charlie"], "Age": [25, 30, 35]}
)
print("Structured Data:")
print(structured_data)
```

```
Structured Data:
   ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35
```

In [3]:
```
unstructured_data = "This is an example of unstructured data. It can be a piece of text, an image, or a video file."
print("\nUnstructured Data:")
print(unstructured_data)
```

```
Unstructured Data:
This is an example of unstructured data. It can be a piece of text, an image, or a video file.
```

In [4]:
```
semi_structured_data = {
    "ID": 1,
    "Name": "Alice",
    "Attributes": {"Height": 165, "Weight": 68},
}
print("\nSemi-structured Data:")
print(semi_structured_data)
```

```
Semi-structured Data:
{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

EXPERIMENT 1D

CONDUCT AN EXPERIMENT TO ENCRYPT AND DECRYPT GIVEN SENSITIVE DATA. USE THE CRYPTOGRAPHY LIBRARY FERNET.

In [1]:
```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher_suite = Fernet(key)
```

In [2]:
```
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqxK-V1khKhlMqhPtiESAX3gxNimHoTDOtzttAQovNVP-WPkxz3q85dHIj0O-
WLJl9NsbMqhe5Pi_rzPWP1SdVqzoZa8n4iJKcFoDnlfwXKbiXnL7gJFOujR5ouehw0MKm5cdP'
```

In [3]:
```
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Decrypted Data: b'Rajalakshmi Engineering College.'
```

EXPERIMENT 2

UPLOAD AND ANALYZE THE DATA SET GIVEN IN CSV FORMAT AND PERFORM DATA PREPROCESSING AND VISUALIZATION.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df = pd.DataFrame(
    {
        "Date": pd.date_range("2023-01-01", periods=16, freq="D"),
        "Product": ["Product A", "Product B", "Product A", "Product C"] * 4,
        "Sales": [
            200,
            150,
            220,
            300,
            180,
            250,
            210,
            340,
            190,
            260,
            230,
            320,
            240,
            280,
            310,
            330,
        ],
        "Quantity": [4, 3, 5, 6, 4, 6, 5, 8, 4, 6, 5, 7, 6, 7, 8, 7],
        "Region": ["North", "South", "North", "East", "West", "North", "South",
"East"]
        * 2,
    }
)
print(df.head())
print(df.isnull().sum())
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
print(df.describe())
```

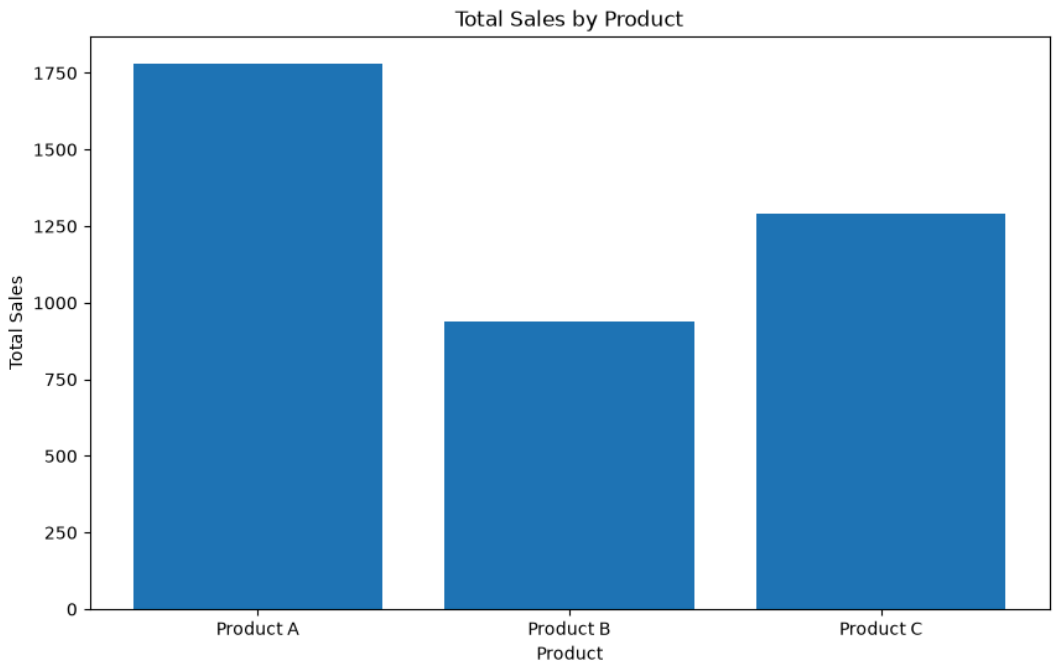
```
        Date    Product  Sales  Quantity Region
0 2023-01-01  Product A    200         4  North
1 2023-01-02  Product B    150         3  South
2 2023-01-03  Product A    220         5  North
3 2023-01-04  Product C    300         6   East
4 2023-01-05  Product A    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
                      Date       Sales   Quantity
count                   16   16.000000  16.000000
mean   2023-01-08 12:00:00  250.625000   5.687500
min    2023-01-01 00:00:00  150.000000   3.000000
25%    2023-01-04 18:00:00  207.500000   4.750000
50%    2023-01-08 12:00:00  245.000000   6.000000
75%    2023-01-12 06:00:00  302.500000   7.000000
max    2023-01-16 00:00:00  340.000000   8.000000
std                    NaN   58.019393   1.493039
```

In [3]:

```python
product_summary = (
    df.groupby("Product").agg({"Sales": "sum", "Quantity": "sum"}).reset_index()
)
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(product_summary["Product"], product_summary["Sales"])
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.title("Total Sales by Product")
plt.show()
```

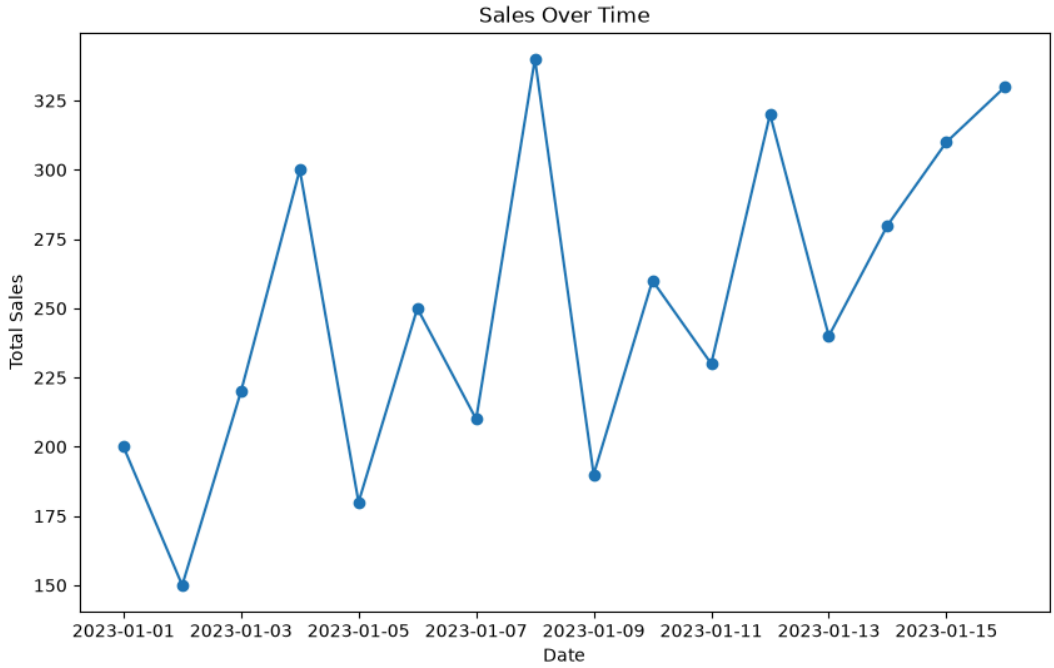
```
     Product  Sales  Quantity
0  Product A   1780        41
1  Product B    940        22
2  Product C   1290        28
```

OUTPUT



In [4]:
```
sales_over_time = df.groupby("Date").agg({"Sales": "sum"}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time["Date"], sales_over_time["Sales"], marker="o")
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.title("Sales Over Time")
plt.show()
```

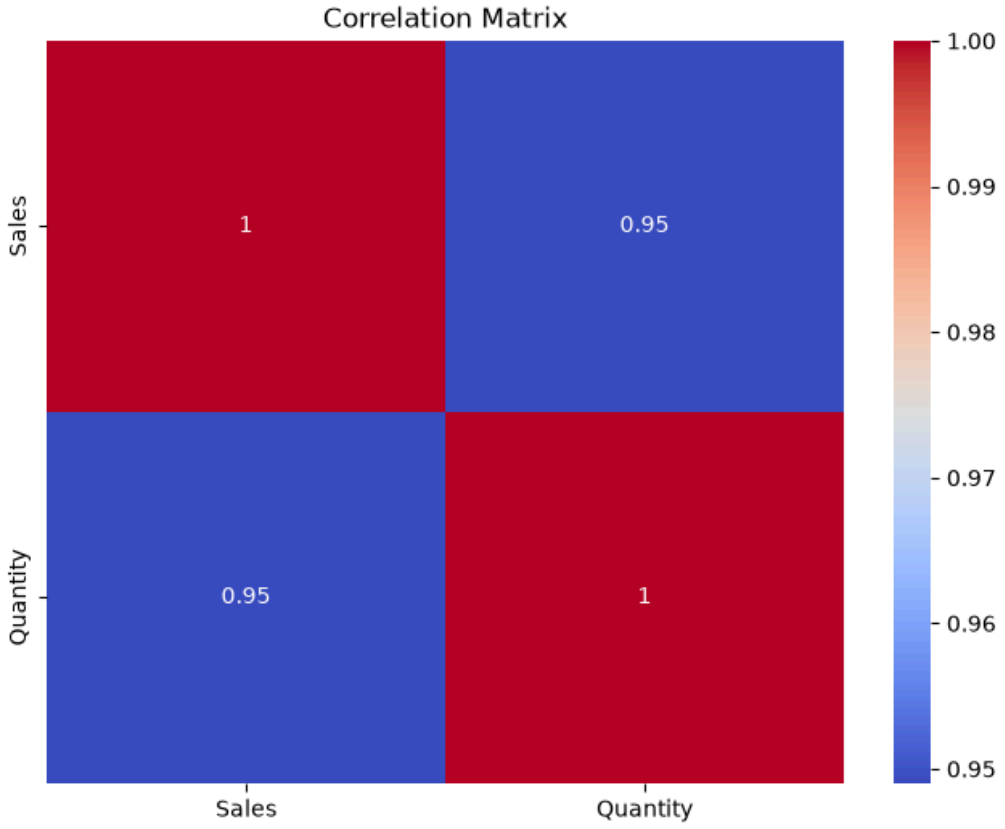
OUTPUT



In [5]:
```
pivot_table = df.pivot_table(
    values="Sales", index="Region", columns="Product", aggfunc="sum", fill_value=0
)
print(pivot_table)
correlation_matrix = df[["Sales", "Quantity"]].corr()
print(correlation_matrix)
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

```
Product  Product A  Product B  Product C
Region
East             0          0       1290
North          840        530          0
South          520        410          0
West           420          0          0
              Sales  Quantity
Sales      1.000000  0.949011
Quantity   0.949011  1.000000
```

OUTPUT


Correlation Matrix
Sales
Quantity
1
0.95
0.95
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

EXPERIMENT 3A

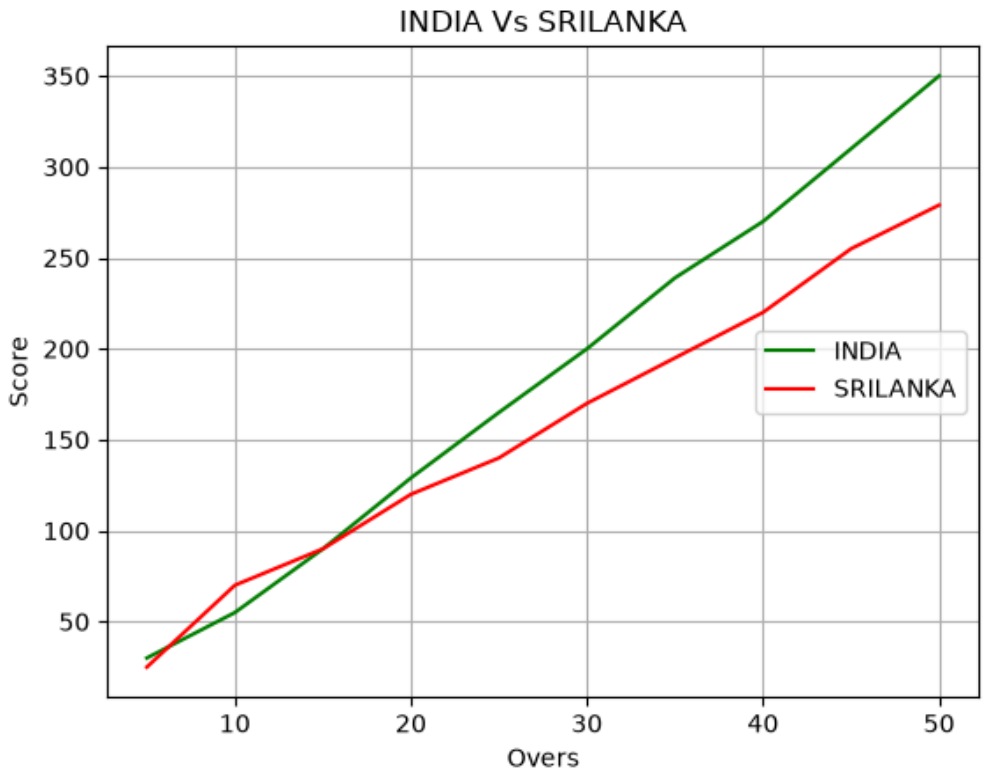
CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING LINE PLOT.

In [1]:
```
import matplotlib.pyplot as plt

Overs = list(range(5, 51, 5))
Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]
Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]
```

In [2]:
```
plt.plot(Overs, Indian_Score, color="green", label="INDIA")
plt.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")
plt.title("INDIA Vs SRILANKA")
plt.xlabel("Overs")
plt.ylabel("Score")
plt.legend(loc="center right")
plt.grid(True)
plt.show()
```

OUTPUT



EXPERIMENT 3B

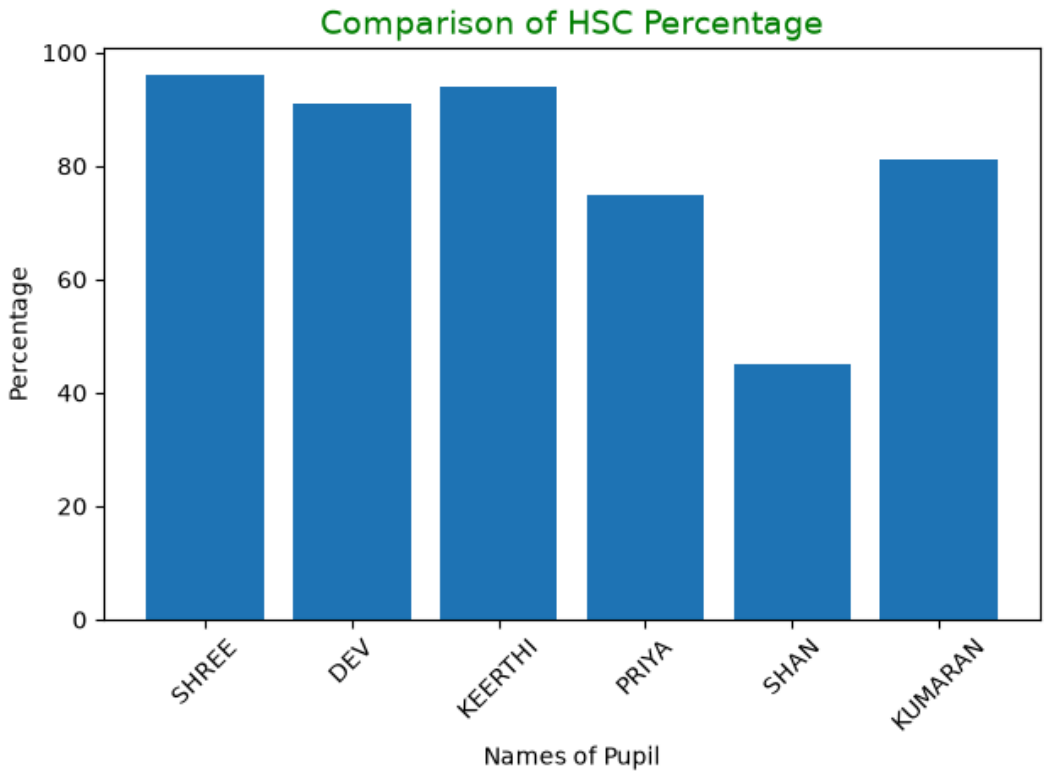
CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING BAR CHART.

In [7]:
```
import matplotlib.pyplot as plt
import numpy as np

Names = ["SHREE", "DEV", "KEERTHI", "PRIYA", "SHAN", "KUMARAN"]
Percentage_hsc = [96, 91, 94, 75, 45, 81]
```

In [8]:
```
plt.bar(Names, Percentage_hsc)
plt.xticks(rotation=45)
plt.xlabel("Names of Pupil")
plt.ylabel("Percentage")
plt.title("Comparison of HSC Percentage", fontsize=14, color="green")
plt.tight_layout()
plt.show()
```

OUTPUT



EXPERIMENT 3C

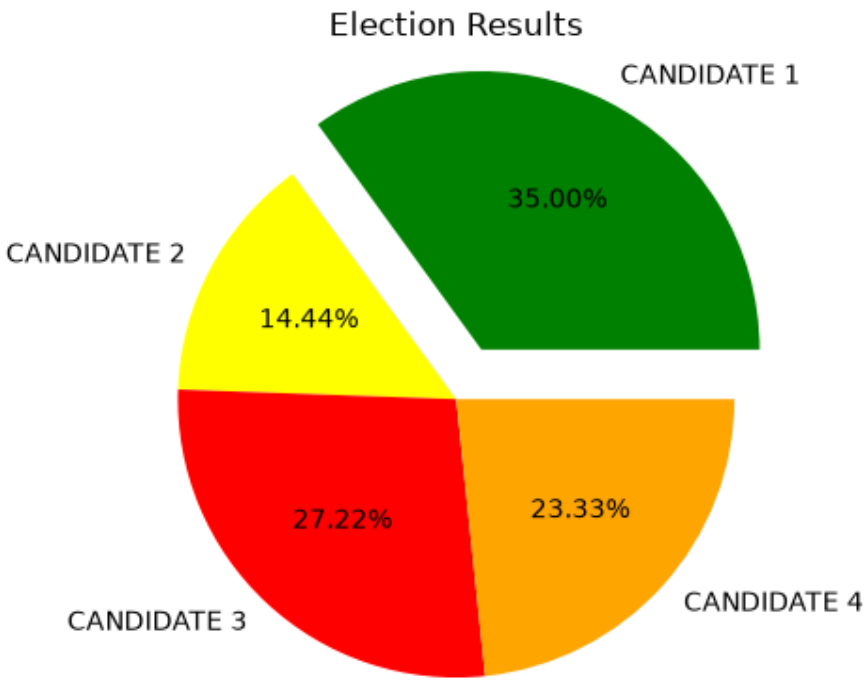
CONDUCT AN EXPERIMENT TO SHOW DATA VISUALIZATION USING PIE CHART.

In [1]:
```
import matplotlib.pyplot as plt

labels = ["CANDIDATE 1", "CANDIDATE 2", "CANDIDATE 3", "CANDIDATE 4"]
Votes = [315, 130, 245, 210]
colors = ["green", "yellow", "red", "orange"]
explode = (0.2, 0, 0, 0)
```

In [2]:
```
plt.pie(Votes, labels=labels, colors=colors, explode=explode, autopct="%0.2f%%")
plt.title("Election Results")
plt.show()
```

OUTPUT



EXPERIMENT 4

TO COUNT THE FREQUENCY OF OCCURRENCE OF A WORD IN A BODY OF TEXT IS OFTEN NEEDED DURING TEXT PROCESSING.

In [1]:
```
from collections import Counter
import re
```

In [2]:
```
text = (
    open(__file__).read()
    if False
    else "Emma by Jane Austen 1816 VOLUME I CHAPTER I Emma Woodhouse handsome clever rich comfortable home happy disposition seemed to unite blessings of existence lived nearly twenty-one years in the world with happiest Emma Emma"
)
tokens = re.findall(r"\w+|[,;\.\[\]]", text)
first50 = tokens[:50]
```

In [3]:
```
freq = Counter(first50)
pairs = [(w, freq[w]) for w in first50]
print("Pairs")
print(pairs)
```

```
Pairs
[('Emma', 4), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 4), ('Woodhouse', 1), ('handsome', 1), ('clever', 1), ('rich', 1), ('comfortable', 1), ('home', 1), ('happy', 1), ('disposition', 1), ('seemed', 1), ('to', 1), ('unite', 1), ('blessings', 1), ('of', 1), ('existence', 1), ('lived', 1), ('nearly', 1), ('twenty', 1), ('one', 1), ('years', 1), ('in', 1), ('the', 1), ('world', 1), ('with', 1), ('happiest', 1), ('Emma', 4), ('Emma', 4)]
```

In [4]:
```
# NLTK version (needs gutenberg + punkt):
# import nltk
# from nltk.tokenize import word_tokenize
# from nltk.corpus import gutenberg
# nltk.download('gutenberg'); nltk.download('punkt'); nltk.download('punkt_tab')
# sample = gutenberg.raw("austen-emma.txt")
# token = word_tokenize(sample)
# wlist = token[:50]
# wordfreq = [wlist.count(w) for w in wlist]
# print(list(zip(wlist, wordfreq)))
```

EXPERIMENT 5

DATA COLLECTION AND INITIAL EXPLORATION

In [13]:
```
import pandas as pd
from sklearn.datasets import load_diabetes

d = load_diabetes(as_frame=True)
df = d.frame
```

In [14]:
```
print(df.head())
print(df.info())
```

```
        age       sex       bmi        bp        s1        s2        s3  \
0  0.038076  0.050680  0.061696  0.021872 -0.044223 -0.034821 -0.043401
1 -0.001882 -0.044642 -0.051474 -0.026328 -0.008449 -0.019163  0.074412
2  0.085299  0.050680  0.044451 -0.005670 -0.045599 -0.034194 -0.032356
3 -0.089063 -0.044642 -0.011595 -0.036656  0.012191  0.024991 -0.036038
4  0.005383 -0.044642 -0.036385  0.021872  0.003935  0.015596  0.008142

         s4        s5        s6  target
0 -0.002592  0.019907 -0.017646   151.0
1 -0.039493 -0.068332 -0.092204    75.0
2 -0.002592  0.002861 -0.025930   141.0
3  0.034309  0.022688 -0.009362   206.0
4 -0.002592 -0.031988 -0.046641   135.0
<class 'pandas.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   s1      442 non-null    float64
 5   s2      442 non-null    float64
 6   s3      442 non-null    float64
 7   s4      442 non-null    float64
 8   s5      442 non-null    float64
 9   s6      442 non-null    float64
 10  target  442 non-null    float64
dtypes: float64(11)
memory usage: 38.1 KB
None
```

In [15]:
```
print(df.describe())
print(df.isnull().sum())
```

```
                age           sex           bmi            bp            s1  \
count  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02
mean  -2.511817e-19  1.230790e-17 -2.245564e-16 -4.797570e-17 -1.381499e-17
std    4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02
min   -1.072256e-01 -4.464164e-02 -9.027530e-02 -1.123988e-01 -1.267807e-01
25%   -3.729927e-02 -4.464164e-02 -3.422907e-02 -3.665608e-02 -3.424784e-02
50%    5.383060e-03 -4.464164e-02 -7.283766e-03 -5.670422e-03 -4.320866e-03
75%    3.807591e-02  5.068012e-02  3.124802e-02  3.564379e-02  2.835801e-02
max    1.107267e-01  5.068012e-02  1.705552e-01  1.320436e-01  1.539137e-01

                s2            s3            s4            s5            s6  \
count  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02  4.420000e+02
mean   3.918434e-17 -5.777179e-18 -9.042540e-18  9.268604e-17  1.130318e-17
std    4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02  4.761905e-02
min   -1.156131e-01 -1.023071e-01 -7.639450e-02 -1.260971e-01 -1.377672e-01
25%   -3.035840e-02 -3.511716e-02 -3.949338e-02 -3.324559e-02 -3.317903e-02
50%   -3.819065e-03 -6.584468e-03 -2.592262e-03 -1.947171e-03 -1.077698e-03
75%    2.984439e-02  2.931150e-02  3.430886e-02  3.243232e-02  2.791705e-02
max    1.987880e-01  1.811791e-01  1.852344e-01  1.335973e-01  1.356118e-01

           target
count  442.000000
mean   152.133484
std     77.093005
min     25.000000
25%     87.000000
50%    140.500000
75%    211.500000
max    346.000000
age       0
sex       0
bmi       0
bp        0
s1        0
s2        0
s3        0
s4        0
s5        0
s6        0
target    0
dtype: int64
```

EXPERIMENT 6

TIME SERIES ANALYSIS OF WEBSITE TRAFFIC

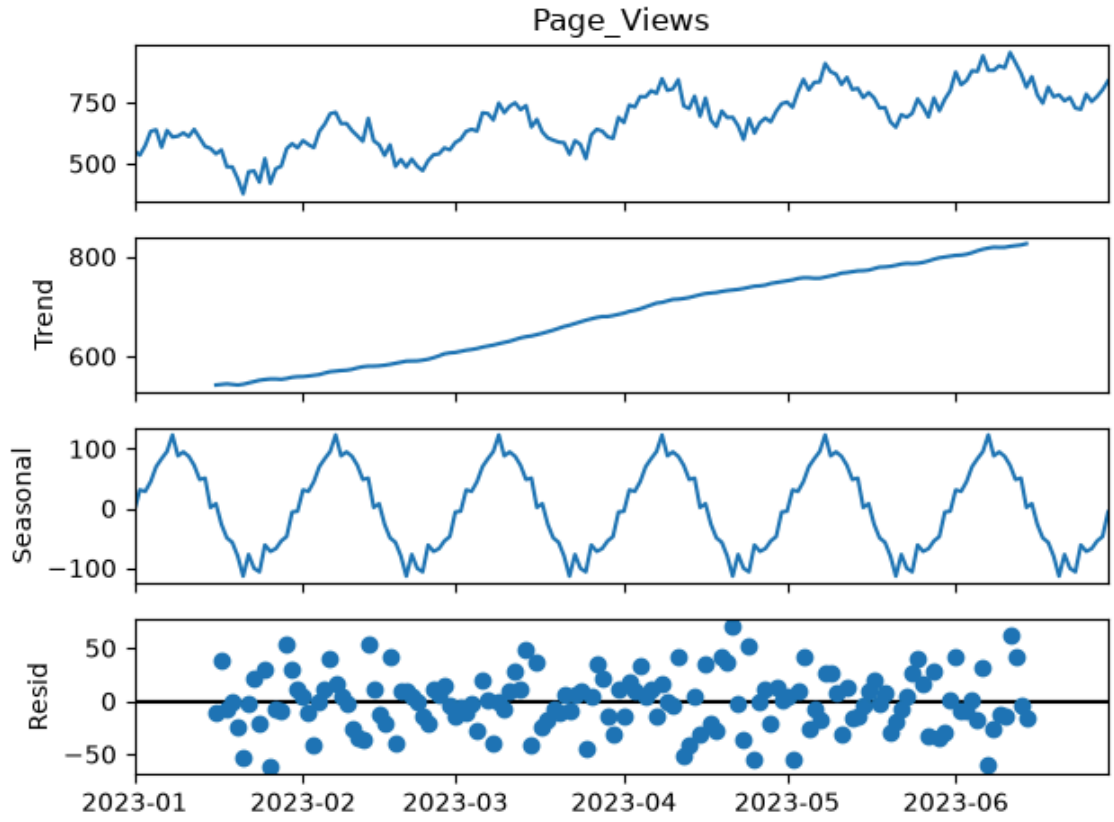
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

In [2]:
```
dates = pd.date_range("2023-01-01", periods=180, freq="D")
np.random.seed(0)
page_views = (
    500
    + np.arange(180) * 2
    + 100 * np.sin(np.arange(180) * 2 * np.pi / 30)
    + np.random.normal(0, 30, 180)
)
unique_visitors = page_views * 0.6 + np.random.normal(0, 20, 180)
bounce_rate = (
    40 + 5 * np.sin(np.arange(180) * 2 * np.pi / 60) + np.random.normal(0, 2, 180)
)
data = pd.DataFrame(
    {
        "Date": dates,
        "Page_Views": page_views,
        "Unique_Visitors": unique_visitors,
        "Bounce_Rate": bounce_rate,
    }
)
data.set_index("Date", inplace=True)
data.iloc[10:12] = (
    data.iloc[10:12].interpolate().fillna(method="bfill").values
    if False
    else data.iloc[10:12]
)
print(data.head())
```

```
            Page_Views  Unique_Visitors  Bounce_Rate
Date
2023-01-01  552.921570       324.673064    41.396914
2023-01-02  534.795885       293.378505    40.530184
2023-01-03  574.035804       331.549114    42.903255
2023-01-04  632.005321       334.735130    42.225015
2023-01-05  638.341222       395.509362    42.002319
```

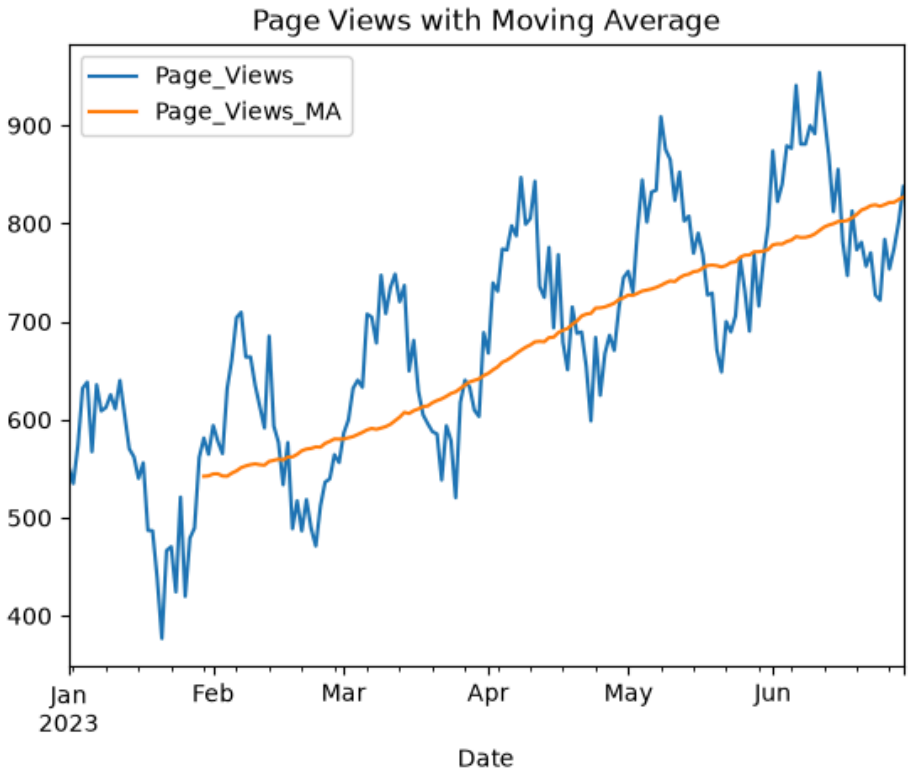
In [3]:
```
decomposition = seasonal_decompose(data["Page_Views"], model="additive", period=30)
decomposition.plot()
plt.show()
```

OUTPUT



In [4]:
```
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
data[["Page_Views", "Page_Views_MA"]].plot()
plt.title("Page Views with Moving Average")
plt.show()
```

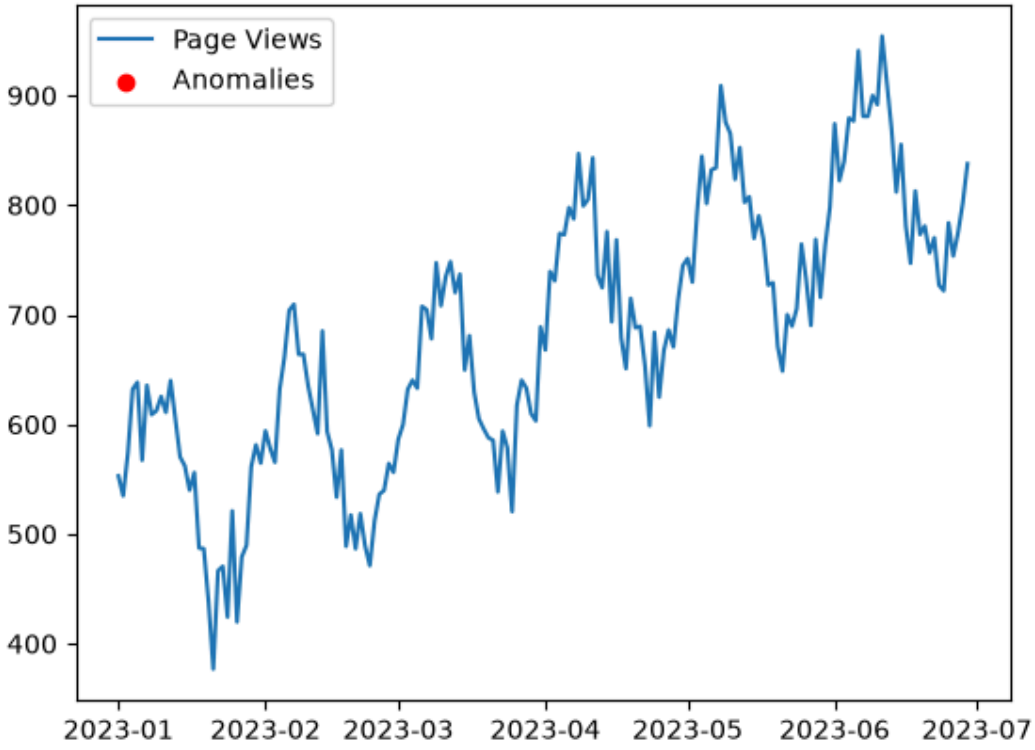
OUTPUT



In [5]:

```
data["Page_Views_Z"] = (data["Page_Views"] - data["Page_Views"].mean()) / data[
    "Page_Views"
].std()
anomalies = data[data["Page_Views_Z"].abs() > 3]
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(anomalies.index, anomalies["Page_Views"], color="red",
label="Anomalies")
plt.legend()
plt.show()
```

OUTPUT



EXPERIMENT 7

RANDOM SAMPLING AND SAMPLING DISTRIBUTION

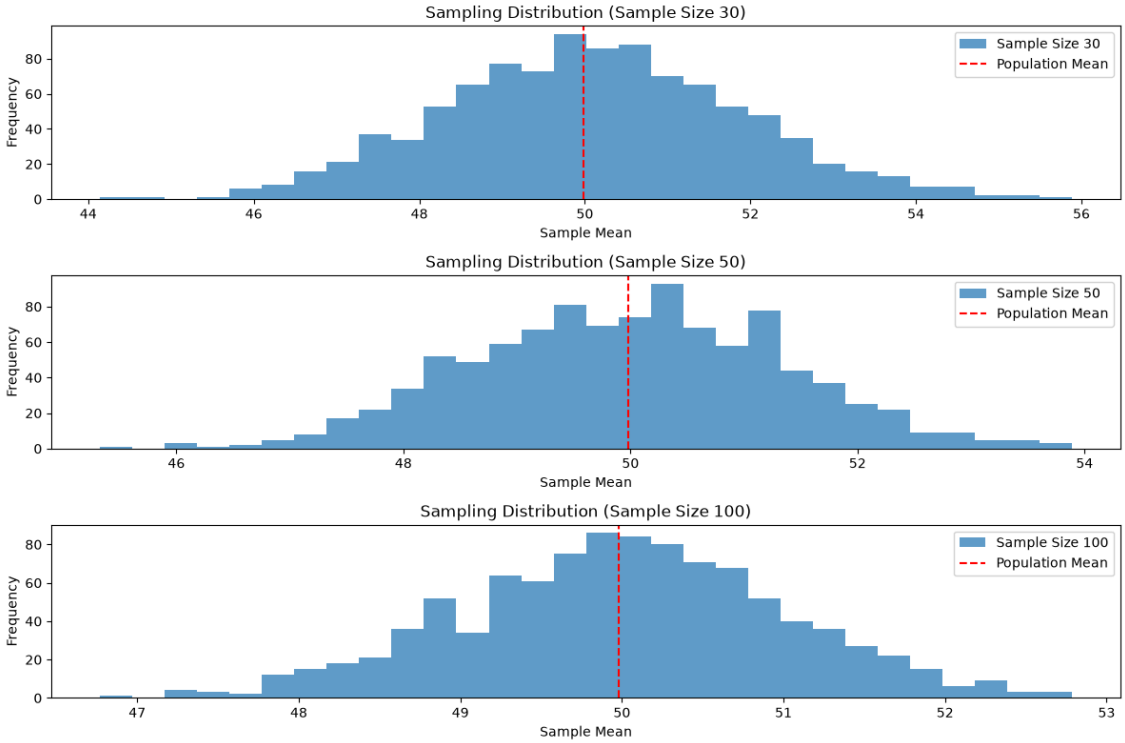
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

population_mean, population_std, population_size = 50, 10, 100000
population = np.random.normal(population_mean, population_std, population_size)
```

In [2]:
```
sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}
for size in sample_sizes:
    sample_means[size] = [
        np.mean(np.random.choice(population, size=size, replace=False))
        for _ in range(num_samples)
    ]
```

In [3]:
```
plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f"Sample Size {size}")
    plt.axvline(
        np.mean(population),
        color="red",
        linestyle="dashed",
        linewidth=1.5,
        label="Population Mean",
    )
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean

EXPERIMENT 8

CONDUCT Z TEST FOR THE DATA GIVEN USING PYTHON

In [ ]:
```
import numpy as np
import scipy.stats as stats

sample_data = np.array(
    [
        152,
        148,
        151,
        149,
        147,
        153,
        150,
        148,
        152,
        149,
        151,
        150,
        149,
        152,
        151,
        148,
        150,
        152,
        149,
        150,
        148,
        153,
        151,
        150,
        149,
        152,
        148,
        151,
        150,
        153,
    ]
)
population_mean = 150
```

In [2]:
```
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
```

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
```

In [3]:
```
alpha = 0.05
if p_value < alpha:
    print(
        "Reject the null hypothesis: The average weight is significantly different
from 150 grams."
    )
else:
    print(
        "Fail to reject the null hypothesis: There is no significant difference in
average weight from 150 grams."
    )
```

```
Fail to reject the null hypothesis: There is no significant difference in average weight from 150
grams.
```

EXPERIMENT 9

EXPERIMENT TO UNDERSTAND MATPLOTLIB LIBRARY USE CASES IN DATA SCIENCE THROUGH VISUALIZATION.

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

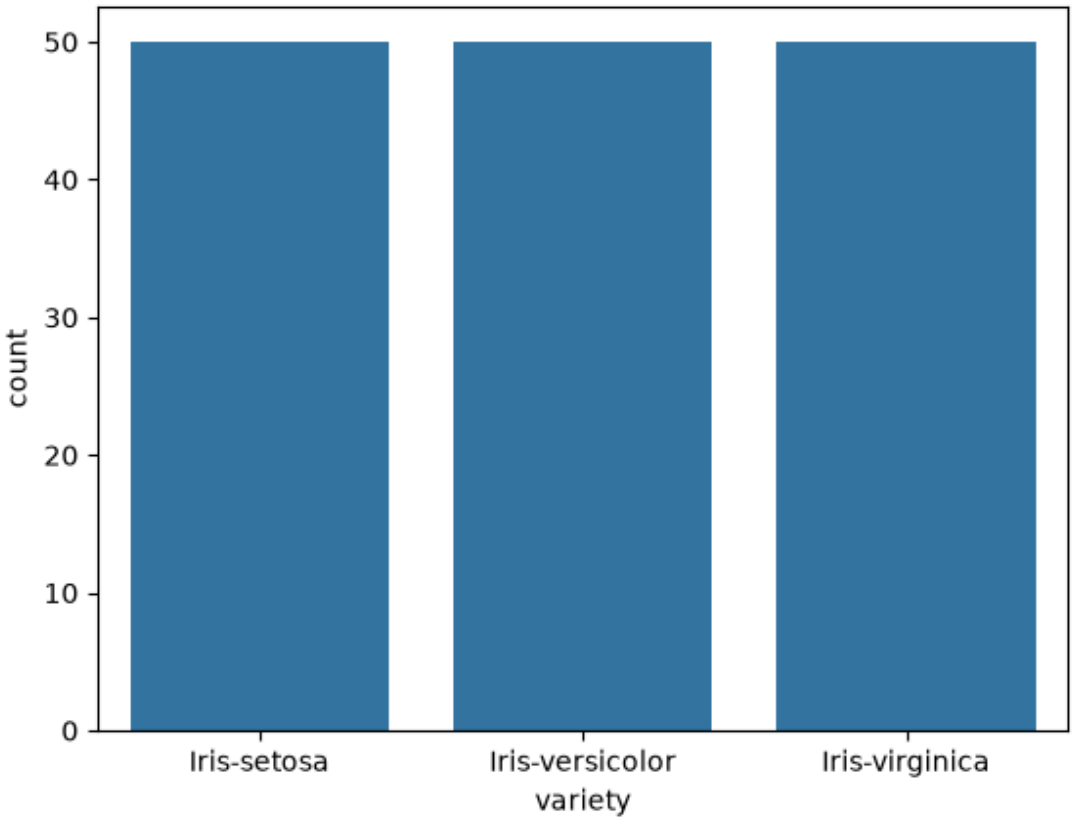
iris = load_iris(as_frame=True)
data = iris.frame
data.rename(
    columns={
        "sepal length (cm)": "SepalLengthCm",
        "sepal width (cm)": "SepalWidthCm",
        "petal length (cm)": "PetalLengthCm",
        "petal width (cm)": "PetalWidthCm",
        "target": "Id",
    },
    inplace=True,
)
data["variety"] = data["Id"].map(
    {0: "Iris-setosa", 1: "Iris-versicolor", 2: "Iris-virginica"}
)
```

In [2]:
```
print(data.head())
print(data.info())
print(data.describe())
print(data["variety"].value_counts())
```

```
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Id      variety
0            5.1           3.5            1.4           0.2   0  Iris-setosa
1            4.9           3.0            1.4           0.2   0  Iris-setosa
2            4.7           3.2            1.3           0.2   0  Iris-setosa
3            4.6           3.1            1.5           0.2   0  Iris-setosa
4            5.0           3.6            1.4           0.2   0  Iris-setosa
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   SepalLengthCm  150 non-null    float64
 1   SepalWidthCm   150 non-null    float64
 2   PetalLengthCm  150 non-null    float64
 3   PetalWidthCm   150 non-null    float64
 4   Id             150 non-null    int64
 5   variety        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
None
       SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm          Id
count     150.000000    150.000000     150.000000    150.000000  150.000000
mean        5.843333      3.057333       3.758000      1.199333    1.000000
std         0.828066      0.435866       1.765298      0.762238    0.819232
min         4.300000      2.000000       1.000000      0.100000    0.000000
25%         5.100000      2.800000       1.600000      0.300000    0.000000
50%         5.800000      3.000000       4.350000      1.300000    1.000000
75%         6.400000      3.300000       5.100000      1.800000    2.000000
max         7.900000      4.400000       6.900000      2.500000    2.000000
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

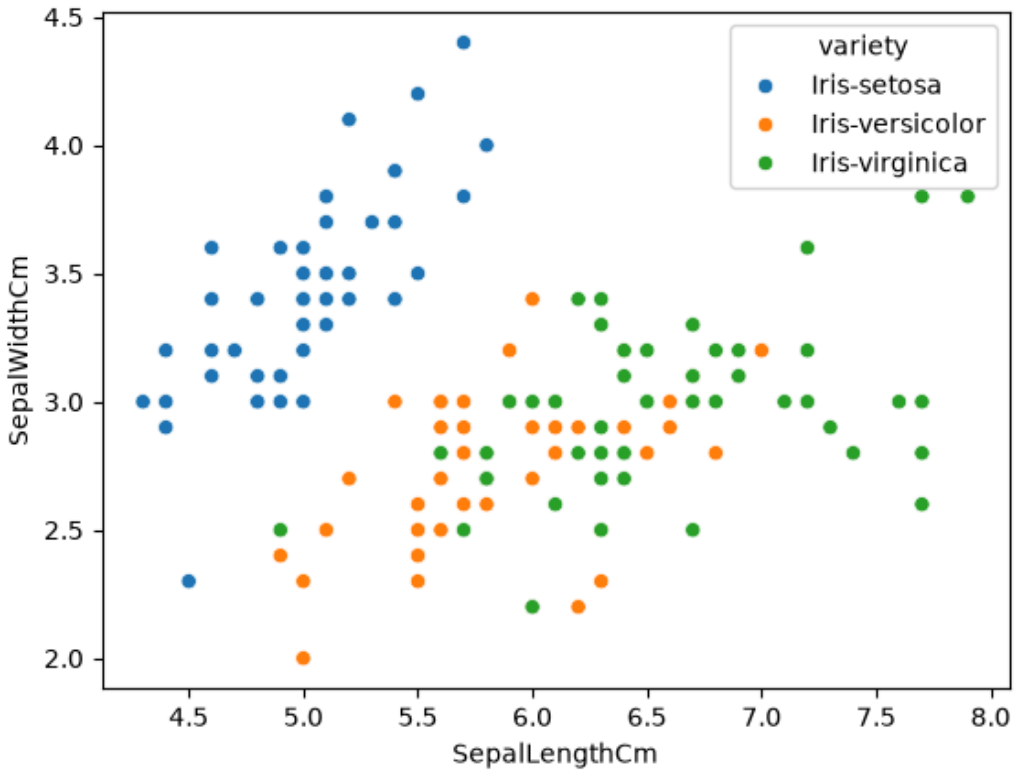
In [3]:
```
sns.countplot(x="variety", data=data)
plt.show()
```

OUTPUT



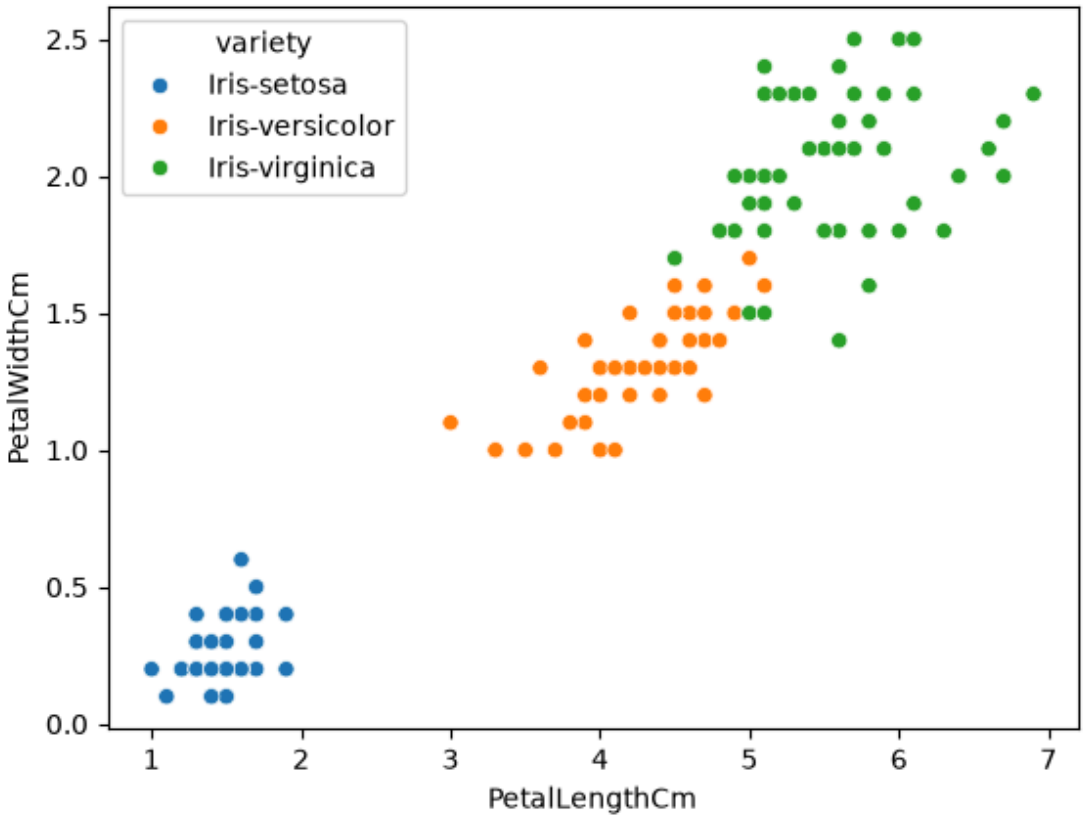
In [4]:
```
sns.scatterplot(x="SepalLengthCm", y="SepalWidthCm", hue="variety", data=data)
plt.show()
```

OUTPUT



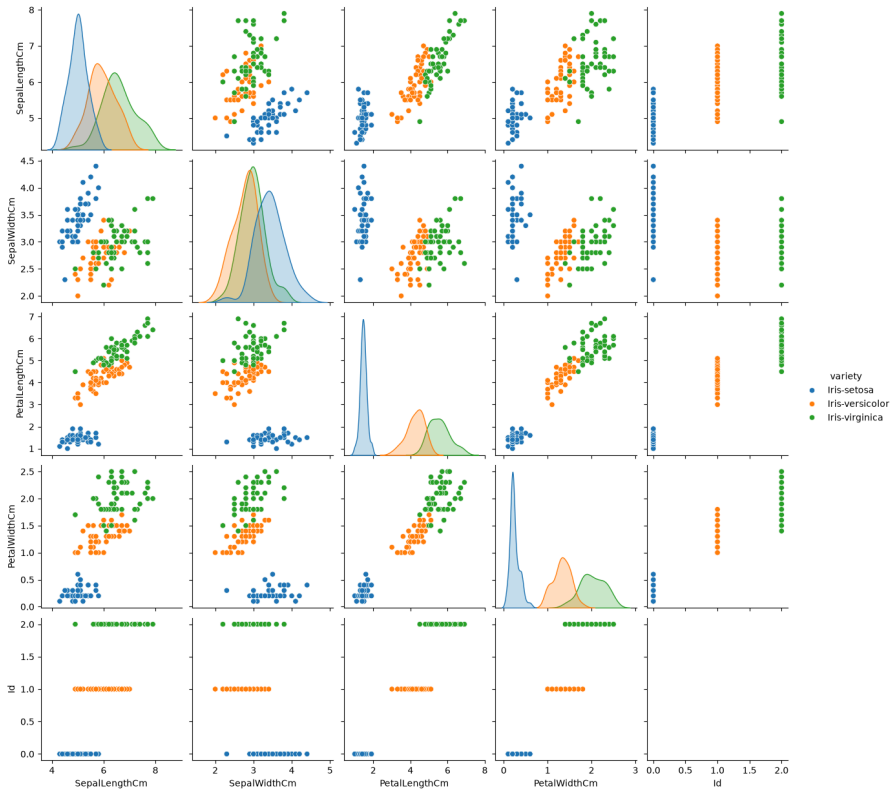
In [5]:
```
sns.scatterplot(x="PetalLengthCm", y="PetalWidthCm", hue="variety", data=data)
plt.show()
```

OUTPUT



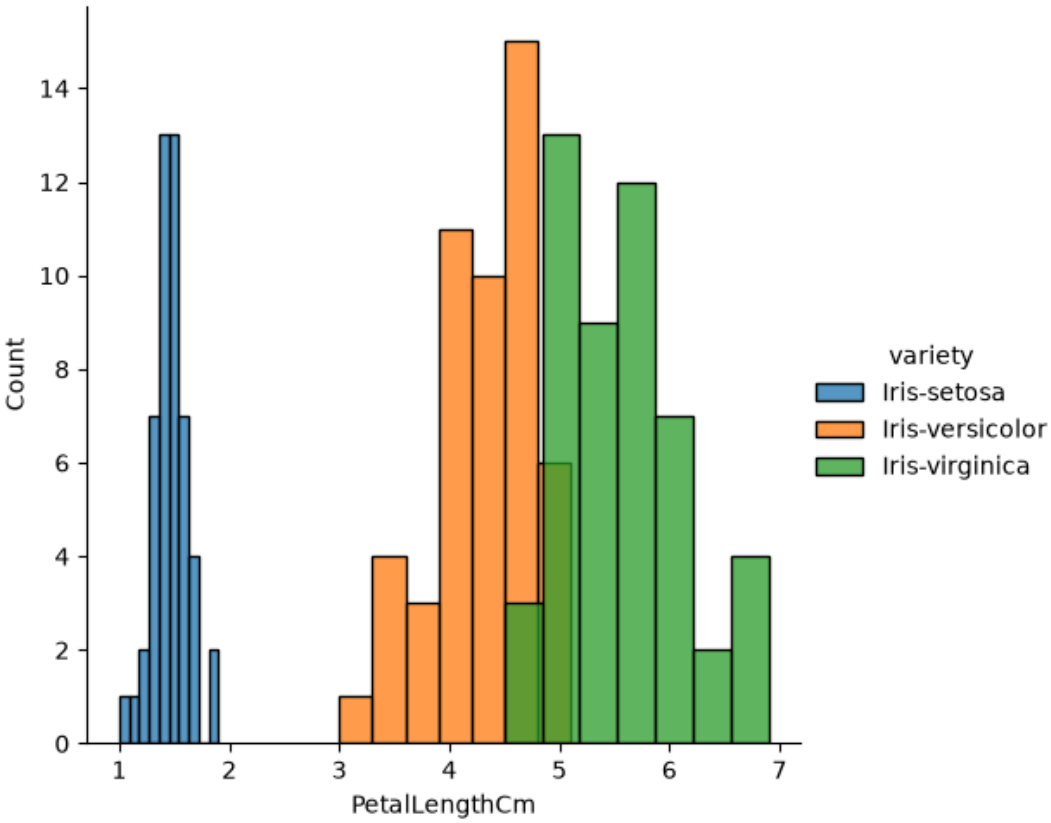
In [6]:
```
sns.pairplot(data, hue="variety")
plt.show()
```

OUTPUT

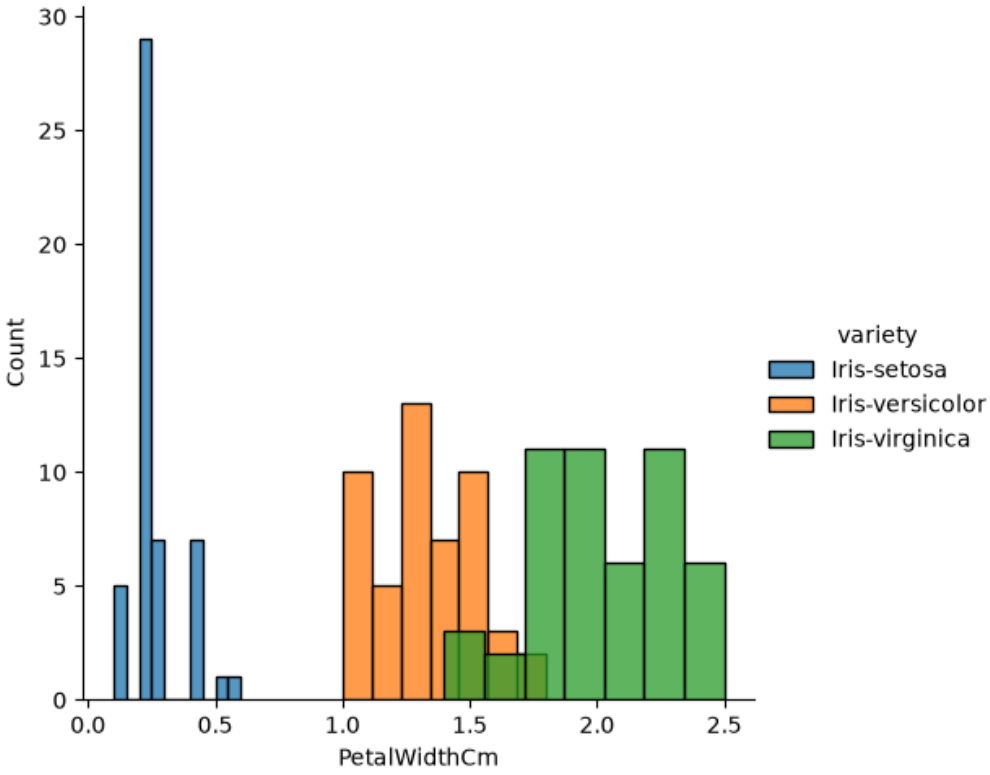


In [7]:
```
for col in ["PetalLengthCm", "PetalWidthCm", "SepalLengthCm", "SepalWidthCm"]:
    sns.FacetGrid(data, hue="variety", height=5).map(sns.histplot, col).add_legend()
    plt.show()
```

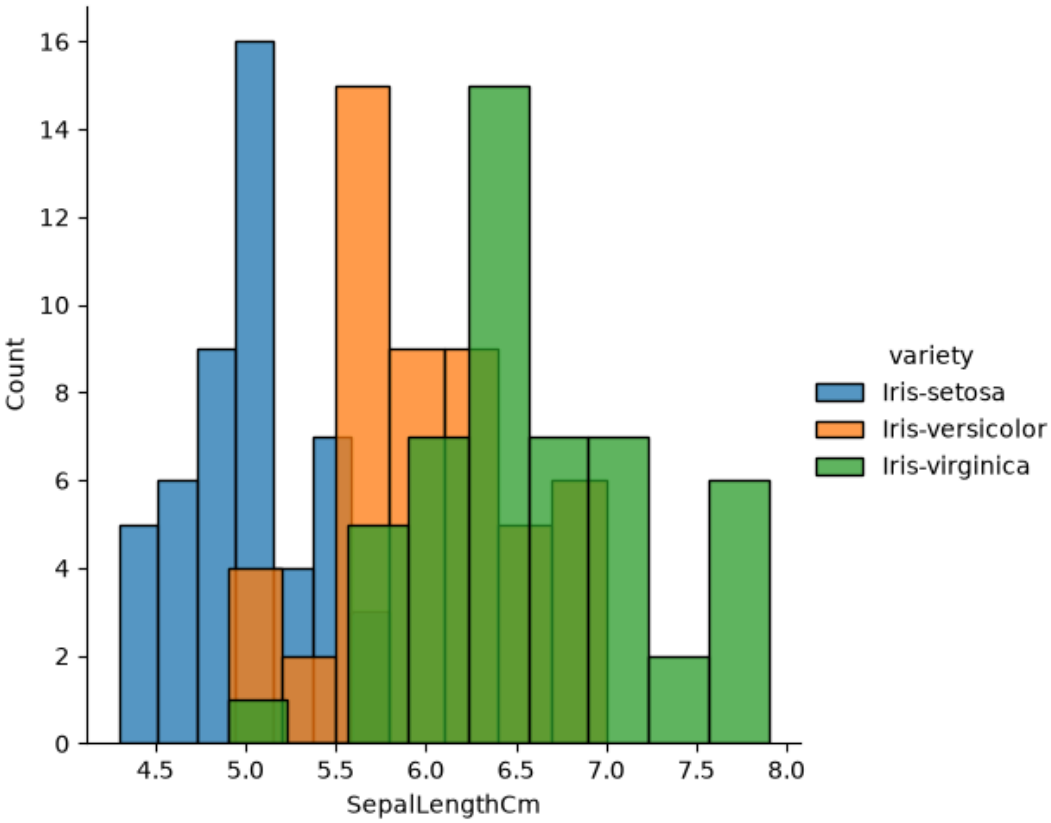
OUTPUT



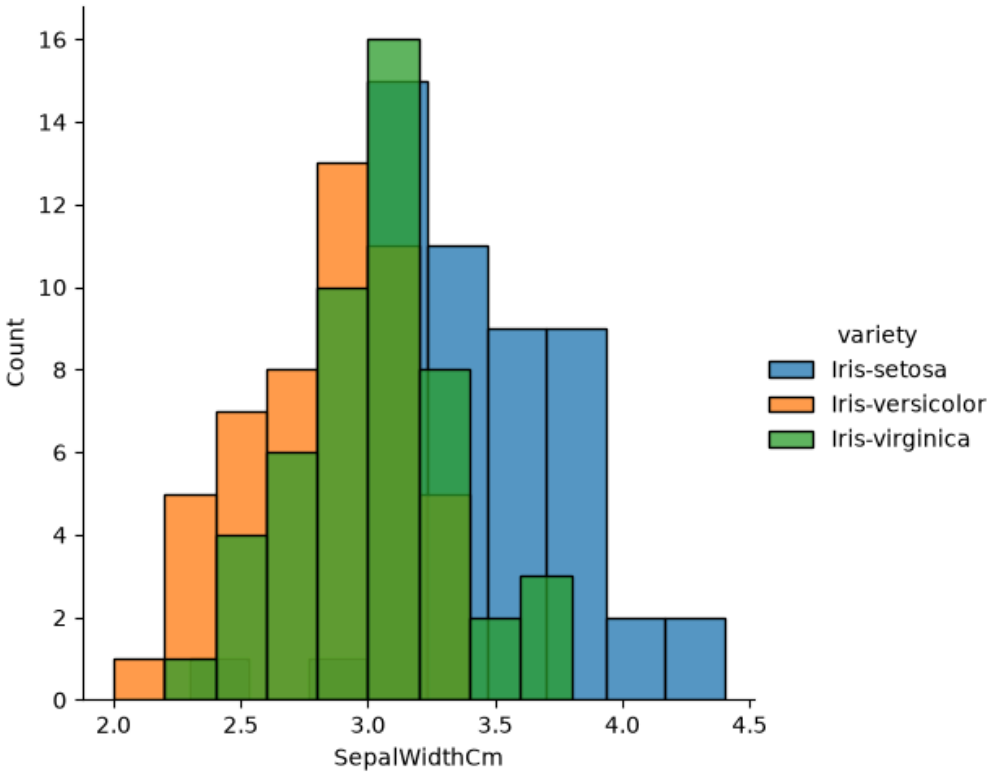
OUTPUT



OUTPUT



OUTPUT



EXPERIMENT 10

EXPERIMENT TO UNDERSTAND ARRAY FUNCTION IN DATA SCIENCE.

In [1]:
```
import numpy as np
```

In [2]:
```
array = np.random.randint(1, 100, 9)
print(array)
print(np.sqrt(array))
print(array.ndim)
```

```
[ 4 39 11 46 35 81 80 36 36]
[2.         6.244998   3.31662479 6.78232998 5.91607978 9.
 8.94427191 6.         6.        ]
1
```

In [3]:
```
new_array = array.reshape(3, 3)
print(new_array)
print(new_array.ndim)
```

```
[[ 4 39 11]
 [46 35 81]
 [80 36 36]]
2
```

In [4]:
```
print(new_array.ravel())
print(new_array[2, 1:3])
print(new_array[1:2, 1:3])
print(new_array[0:2, 0:1])
print(new_array[0:3, 0:1])
print(new_array[1:3])
```

```
[ 4 39 11 46 35 81 80 36 36]
[36 36]
[[35 81]]
[[ 4]
 [46]]
[[ 4]
 [46]
 [80]]
[[46 35 81]
 [80 36 36]]
```

EXPERIMENT 11

EXPERIMENT TO UNDERSTAND PANDAS LIBRARY USE CASES IN DATA SCIENCE.

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df = pd.DataFrame(
    [[1, "Smith", 50000], [2, "Jones", 60000]], columns=["EmpId", "Name", "Salary"]
)
print(df)
```

```
   EmpId   Name  Salary
0      1  Smith   50000
1      2  Jones   60000
```

In [3]:
```
print(df.info())
print(df.describe())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   EmpId   2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 180.0 bytes
None
          EmpId        Salary
count  2.000000      2.000000
mean   1.500000  55000.000000
std    0.707107   7071.067812
min    1.000000  50000.000000
25%    1.250000  52500.000000
50%    1.500000  55000.000000
75%    1.750000  57500.000000
max    2.000000  60000.000000
```

In [4]:
```
print(df["Salary"].mean(), df["Salary"].median(), df["Salary"].mode().tolist())
print(df["Salary"].var(), df["Salary"].std())
```

```
55000.0 55000.0 [50000, 60000]
50000000.0 7071.067811865475
```

In [5]:
```
print(df.columns.tolist())
print(df.values)
```

```
['EmpId', 'Name', 'Salary']
[[1 'Smith' 50000]
 [2 'Jones' 60000]]
```

EXPERIMENT 12

EXPERIMENT TO DETECT OUTLIERS IN A GIVEN DATA SET.

In [1]:
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

In [2]:
```
np.random.seed(0)
array = np.random.randint(1, 100, 16)
print(array)
print("mean:", array.mean())
print(
    "Q1:",
    np.percentile(array, 25),
    "Median:",
    np.percentile(array, 50),
    "Q3:",
    np.percentile(array, 75),
)
```

```
[45 48 65 68 68 10 84 22 37 88 71 89 89 13 59 66]
mean: 57.625
Q1: 43.0 Median: 65.5 Q3: 74.25
```

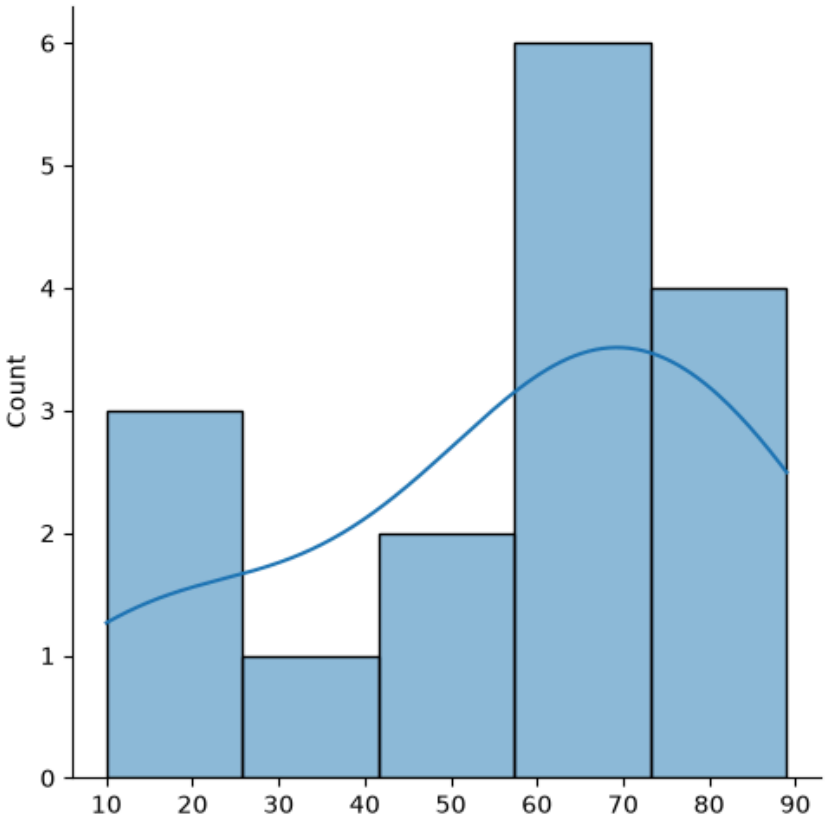
In [3]:
```
def outDetection(a):
    Q1, Q3 = np.percentile(a, [25, 75])
    IQR = Q3 - Q1
    return Q1 - 1.5 * IQR, Q3 + 1.5 * IQR


lr, ur = outDetection(array)
print(lr, ur)
```

```
-3.875 121.125
```

In [4]:
```
sns.displot(array, kde=True)
plt.show()
```

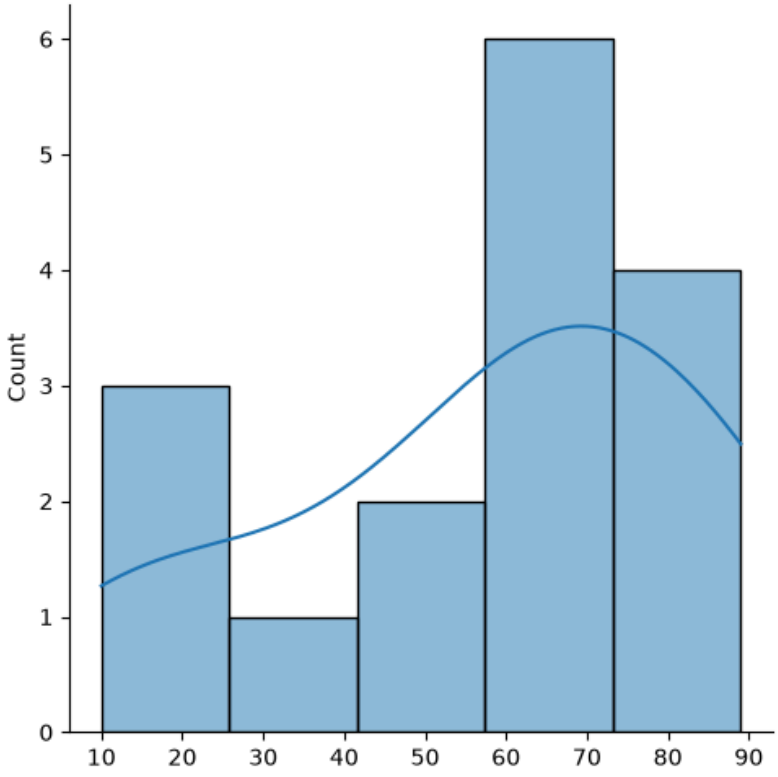
OUTPUT



In [5]:
```
new_array = array[(array > lr) & (array < ur)]
print(new_array)
sns.displot(new_array, kde=True)
plt.show()
```

```
[45 48 65 68 68 10 84 22 37 88 71 89 89 13 59 66]
```

OUTPUT



EXPERIMENT 13

EXPERIMENT TO HANDLE MISSING VALUES AND INAPPROPRIATE DATA IN A GIVEN DATA SET.

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:

```
df = pd.DataFrame(
    {
        "CustomerID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 10],
        "Age_Group": [
            "20-25",
            "30-35",
            "25-30",
            "20-25",
            "35+",
            "35+",
            "35+",
            "20-25",
            "25-30",
            "25-30",
            "30-35",
        ],
        "Rating(1-5)": [4, 5, 6, -1, 3, 3, 4, 7, 2, 2, 5],
        "Hotel": [
            "Ibis",
            "LemonTree",
            "RedFox",
            "LemonTree",
            "Ibis",
            "Ibys",
            "RedFox",
            "LemonTree",
            "Ibis",
            "Ibis",
            "RedFox",
        ],
        "FoodPreference": [
            "veg",
            "Non-Veg",
            "Veg",
            "Veg",
            "Vegetarian",
            "Non-Veg",
            "Vegetarian",
            "Veg",
            "Non-Veg",
            "Non-Veg",
            "non-Veg",
        ],
        "Bill": [1300, 2000, 1322, 1234, 989, 1909, 1000, 2999, 3456, 3456, 6755],
        "NoOfPax": [2, 3, 2, 2, 2, 2, -1, -10, 3, 3, 4],
        "EstimatedSalary": [
            40000,
            59000,
            30000,
            120000,
            45000,
            122220,
            21122,
            345673,
            -99999,
            -99999,
            87777,
```

```
        ],
    }
)
print(df)
```

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0           1     20-25            4       Ibis            veg  1300
1           2     30-35            5  LemonTree        Non-Veg  2000
2           3     25-30            6    RedFox            Veg  1322
3           4     20-25           -1  LemonTree            Veg  1234
4           5       35+            3       Ibis     Vegetarian   989
5           6       35+            3       Ibys        Non-Veg  1909
6           7       35+            4    RedFox     Vegetarian  1000
7           8     20-25            7  LemonTree            Veg  2999
8           9     25-30            2       Ibis        Non-Veg  3456
9           9     25-30            2       Ibis        Non-Veg  3456
10         10     30-35            5    RedFox        non-Veg  6755

    NoOfPax  EstimatedSalary
0         2           40000
1         3           59000
2         2           30000
3         2          120000
4         2           45000
5         2          122220
6        -1           21122
7       -10          345673
8         3          -99999
9         3          -99999
10        4           87777
```

In [3]:
```
print(df.duplicated())
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)
```

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

In [4]:
```
df.loc[df["CustomerID"] < 0, "CustomerID"] = np.nan
df.loc[df["Bill"] < 0, "Bill"] = np.nan
df.loc[df["EstimatedSalary"] < 0, "EstimatedSalary"] = np.nan
df.loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20), "NoOfPax"] = np.nan
df.loc[(df["Rating(1-5)"] < 1) | (df["Rating(1-5)"] > 5), "Rating(1-5)"] = np.nan
```

In [5]:
```
df["Hotel"].replace(["Ibys"], "Ibis", inplace=True)
df["FoodPreference"].replace(["Vegetarian", "veg"], "Veg", inplace=True)
df["FoodPreference"].replace(["non-Veg"], "Non-Veg", inplace=True)
```

```
C:\Users\techn\AppData\Local\Temp\ipykernel_12052\911984210.py:1: ChainedAssignmentError: A value
is being set on a copy of a DataFrame or Series through chained assignment using an inplace
method.
Such inplace method never works to update the original DataFrame or Series, because the
intermediate object on which we are setting values always behaves as a copy (due to Copy-on-
Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid
an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/copy_on_write.html
  df["Hotel"].replace(["Ibys"], "Ibis", inplace=True)
C:\Users\techn\AppData\Local\Temp\ipykernel_12052\911984210.py:2: ChainedAssignmentError: A value
is being set on a copy of a DataFrame or Series through chained assignment using an inplace
method.
Such inplace method never works to update the original DataFrame or Series, because the
intermediate object on which we are setting values always behaves as a copy (due to Copy-on-
Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid
an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/copy_on_write.html
  df["FoodPreference"].replace(["Vegetarian", "veg"], "Veg", inplace=True)
C:\Users\techn\AppData\Local\Temp\ipykernel_12052\911984210.py:3: ChainedAssignmentError: A value
is being set on a copy of a DataFrame or Series through chained assignment using an inplace
method.
Such inplace method never works to update the original DataFrame or Series, because the
intermediate object on which we are setting values always behaves as a copy (due to Copy-on-
Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid
an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/copy_on_write.html
  df["FoodPreference"].replace(["non-Veg"], "Non-Veg", inplace=True)
```

Out[5]:
```
0          veg
1      Non-Veg
2          Veg
3          Veg
4    Vegetarian
5      Non-Veg
6    Vegetarian
7          Veg
8      Non-Veg
9      Non-Veg
Name: FoodPreference, dtype: str
```

In [6]:
```
df["EstimatedSalary"] = df["EstimatedSalary"].fillna(
    round(df["EstimatedSalary"].mean())
)
df["NoOfPax"] = df["NoOfPax"].fillna(round(df["NoOfPax"].median()))
df["Rating(1-5)"] = df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()))
df["Bill"] = df["Bill"].fillna(round(df["Bill"].mean()))
print(df)
print(df.info())
```

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25          4.0       Ibis            veg  1300.0
1         2.0     30-35          5.0  LemonTree        Non-Veg  2000.0
2         3.0     25-30          4.0     RedFox            Veg  1322.0
3         4.0     20-25          4.0  LemonTree            Veg  1234.0
4         5.0       35+          3.0       Ibis     Vegetarian   989.0
5         6.0       35+          3.0       Ibys        Non-Veg  1909.0
6         7.0       35+          4.0     RedFox     Vegetarian  1000.0
7         8.0     20-25          4.0  LemonTree            Veg  2999.0
8         9.0     25-30          2.0       Ibis        Non-Veg  3456.0
9        10.0     30-35          5.0     RedFox        non-Veg  6755.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       10 non-null     float64
 1   Age_Group        10 non-null     str
 2   Rating(1-5)      10 non-null     float64
 3   Hotel            10 non-null     str
 4   FoodPreference   10 non-null     str
 5   Bill             10 non-null     float64
 6   NoOfPax          10 non-null     float64
 7   EstimatedSalary  10 non-null     float64
dtypes: float64(5), str(3)
memory usage: 772.0 bytes
None
```

EXPERIMENT 14

EXPERIMENT TO UNDERSTAND FEATURE SCALING

In [ ]:
```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler
```

In [ ]:
```
df = pd.DataFrame(
    {
        "Country": [
            "France",
            "Spain",
            "Germany",
            "Spain",
            "Germany",
            "France",
            "Spain",
            "France",
            None,
            "France",
        ],
        "Age": [44, 27, 30, 38, 40, 35, None, 48, 50, 37],
        "Salary": [72000, 48000, 54000, 61000, None, 58000, 52000, 79000, 83000,
67000],
        "Purchased": ["No", "Yes", "No", "No", "Yes", "Yes", "No", "Yes", "No",
"Yes"],
    }
)
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
```

In [ ]:
```
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
age_imp = SimpleImputer(strategy="mean")
sal_imp = SimpleImputer(strategy="mean")
features[:, [1]] = age_imp.fit_transform(features[:, [1]])
features[:, [2]] = sal_imp.fit_transform(features[:, [2]])
print(features)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['France' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

In [ ]:
```
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
print(Country)
```

```
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]]
```

In [ ]:
```
final_set = np.concatenate((Country, features[:, [1, 2]].astype(float)), axis=1)
print(final_set)
```

```
[[1.00000000e+00 0.00000000e+00 0.00000000e+00 4.40000000e+01
  7.20000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 2.70000000e+01
  4.80000000e+04]
 [0.00000000e+00 1.00000000e+00 0.00000000e+00 3.00000000e+01
  5.40000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 3.80000000e+01
  6.10000000e+04]
 [0.00000000e+00 1.00000000e+00 0.00000000e+00 4.00000000e+01
  6.37777778e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 3.50000000e+01
  5.80000000e+04]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00 3.87777778e+01
  5.20000000e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 4.80000000e+01
  7.90000000e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 5.00000000e+01
  8.30000000e+04]
 [1.00000000e+00 0.00000000e+00 0.00000000e+00 3.70000000e+01
  6.70000000e+04]]
```

In [ ]:
```
sc = StandardScaler()
print(sc.fit_transform(final_set))
```

```
[[ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-1.00000000e+00  2.00000000e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-1.00000000e+00  2.00000000e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-1.00000000e+00 -5.00000000e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  1.34013983e+00
   1.38753832e+00]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01  1.63077256e+00
   1.75214693e+00]
 [ 1.00000000e+00 -5.00000000e-01 -6.54653671e-01 -2.58340208e-01
   2.93712492e-01]]
```

In [ ]:
```
mms = MinMaxScaler(feature_range=(0, 1))
print(mms.fit_transform(final_set))
```

```
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
 [0.         1.         0.         0.56521739 0.45079365]
 [1.         0.         0.         0.34782609 0.28571429]
 [0.         0.         1.         0.51207729 0.11428571]
 [1.         0.         0.         0.91304348 0.88571429]
 [1.         0.         0.         1.         1.        ]
 [1.         0.         0.         0.43478261 0.54285714]]
```

EXPERIMENT 15

EXPERIMENT TO UNDERSTAND THE DATA PREPROCESSING IN DATA SCIENCE

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df = pd.DataFrame(
    {
        "Country": [
            "France",
            "Spain",
            "Germany",
            "Spain",
            "Germany",
            "France",
            "Spain",
            "France",
            None,
            "France",
        ],
        "Age": [44, 27, 30, 38, 40, 35, None, 48, None, 37],
        "Salary": [72000, 48000, 54000, 61000, None, 58000, 52000, 79000, 83000,
67000],
        "Purchased": ["No", "Yes", "No", "No", "Yes", "Yes", "No", "Yes", "No",
"Yes"],
    }
)
print(df)
print(df.info())
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN   NaN  83000.0        No
9   France  37.0  67000.0       Yes
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      str
 1   Age        8 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
None
```

In [3]:
```
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(round(df["Salary"].mean()))
print(df)
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  37.5  52000.0        No
7   France  48.0  79000.0       Yes
8   France  37.5  83000.0        No
9   France  37.0  67000.0       Yes
```

In [4]:
```
print(pd.get_dummies(df["Country"]))
```

```
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8    True    False  False
9    True    False  False
```

In [5]:
```
updated_dataset = pd.concat(
    [pd.get_dummies(df["Country"]), df.iloc[:, [1, 2, 3]]], axis=1
)
updated_dataset["Purchased"].replace(["No", "Yes"], [0, 1], inplace=True)
print(updated_dataset)
```

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  37.5  52000.0        No
7    True    False  False  48.0  79000.0       Yes
8    True    False  False  37.5  83000.0        No
9    True    False  False  37.0  67000.0       Yes
```

```
C:\Users\techn\AppData\Local\Temp\ipykernel_7248\4031935903.py:4: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  updated_dataset["Purchased"].replace(["No", "Yes"], [0, 1], inplace=True)
```

EXPERIMENT 16

EXPERIMENT TO UNDERSTAND EDA-QUANTITATIVE AND QUALITATIVE ANALYSIS.

In [1]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

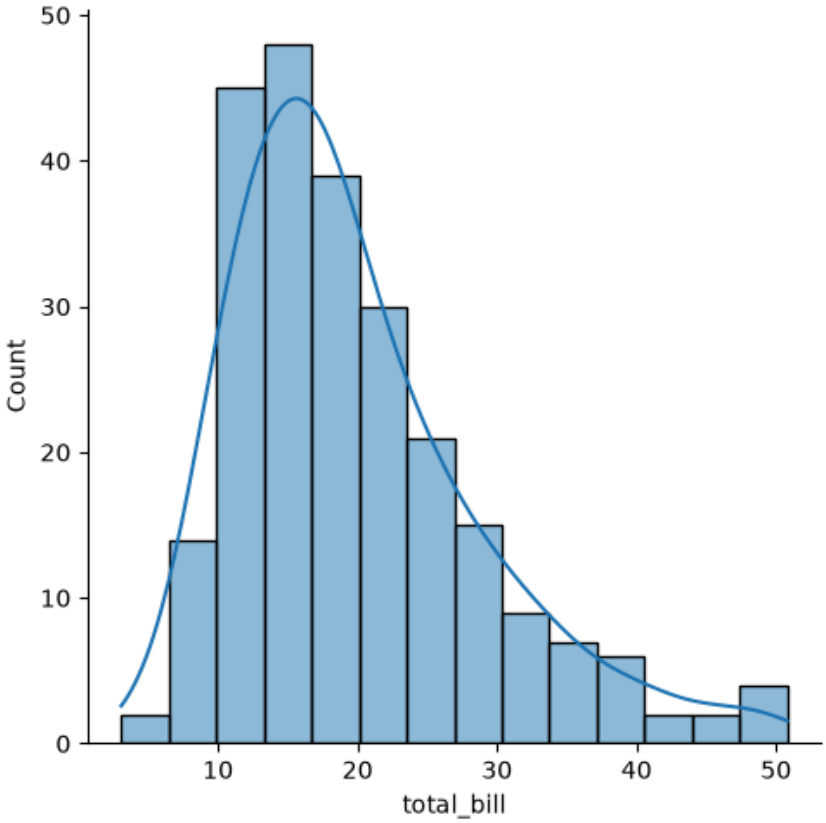
try:
    tips = sns.load_dataset("tips")
except Exception:
    import numpy as np

    n = 244
    tips = pd.DataFrame(
        {
            "total_bill": np.random.normal(20, 8, n).clip(3, 60),
            "tip": np.random.normal(3, 1.5, n).clip(1, 10),
            "sex": np.random.choice(["Male", "Female"], n),
            "smoker": np.random.choice(["Yes", "No"], n),
            "day": np.random.choice(["Thur", "Fri", "Sat", "Sun"], n),
            "time": np.random.choice(["Lunch", "Dinner"], n),
            "size": np.random.randint(1, 6, n),
        }
    )
print(tips.head())
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

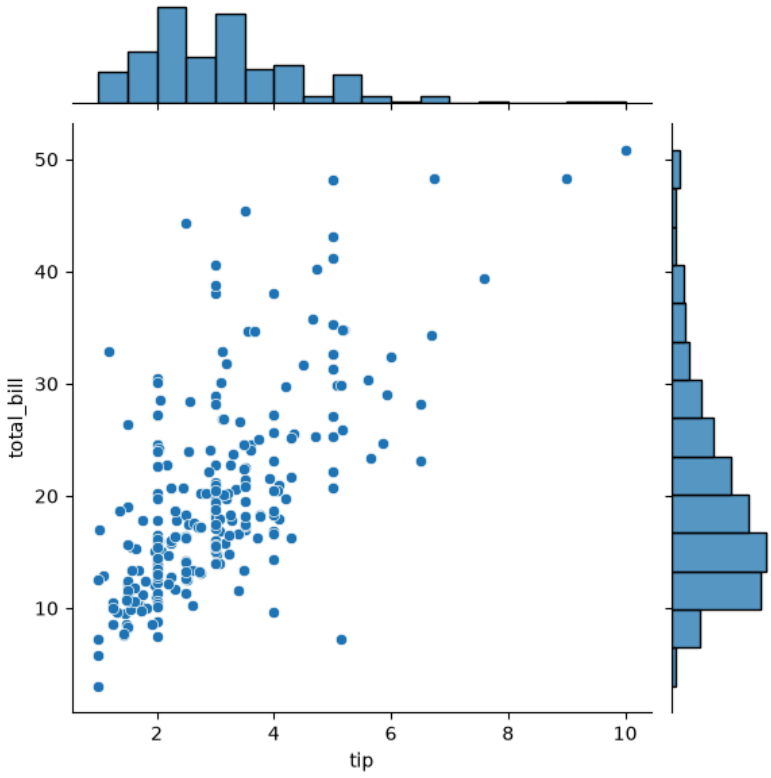
In [2]:
```
sns.displot(tips["total_bill"], kde=True)
plt.show()
```

OUTPUT



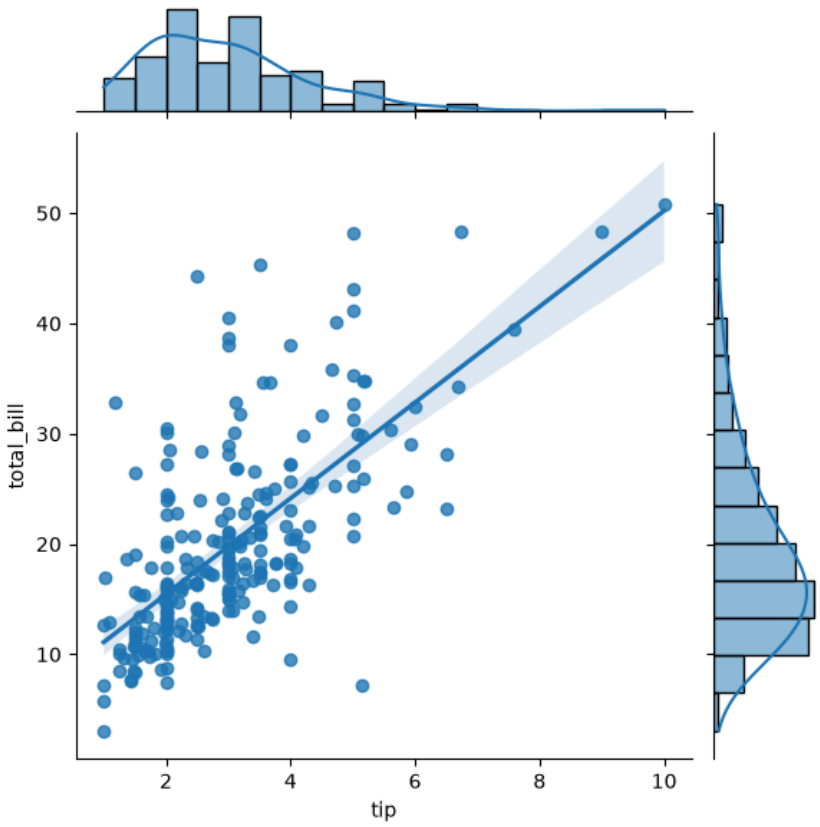
In [3]:
```
sns.jointplot(x="tip", y="total_bill", data=tips)
plt.show()
```

OUTPUT



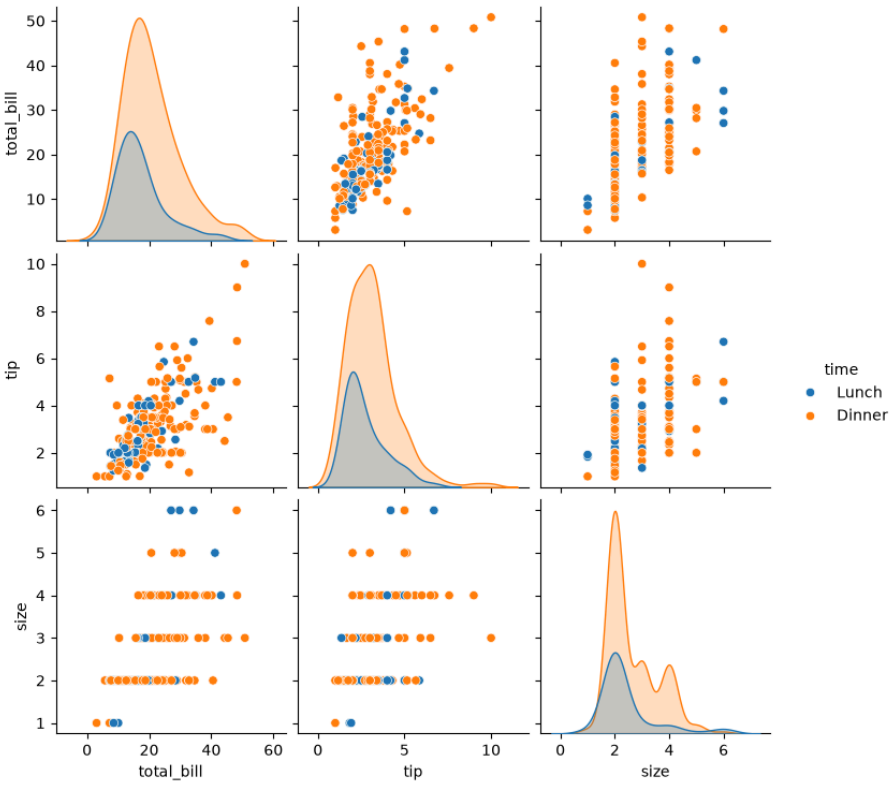
In [4]:
```
sns.jointplot(x="tip", y="total_bill", data=tips, kind="reg")
plt.show()
```

OUTPUT



In [5]:
```
sns.pairplot(tips, hue="time")
plt.show()
```

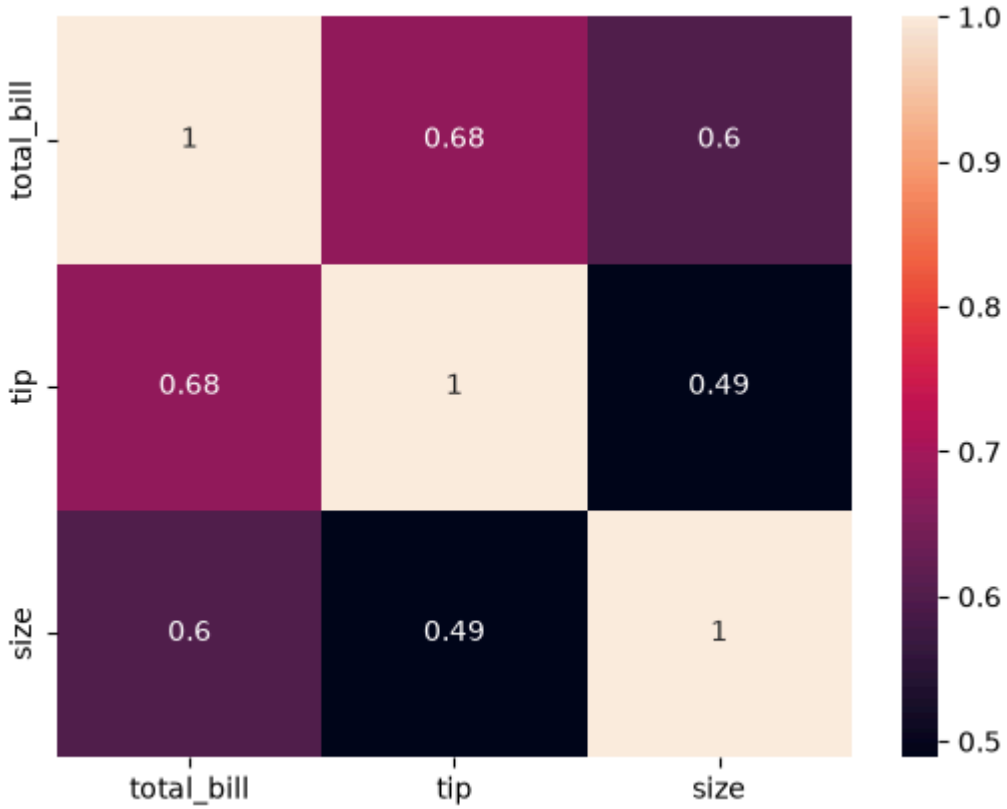
OUTPUT



In [6]:
```
print(tips.corr(numeric_only=True))
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()
```

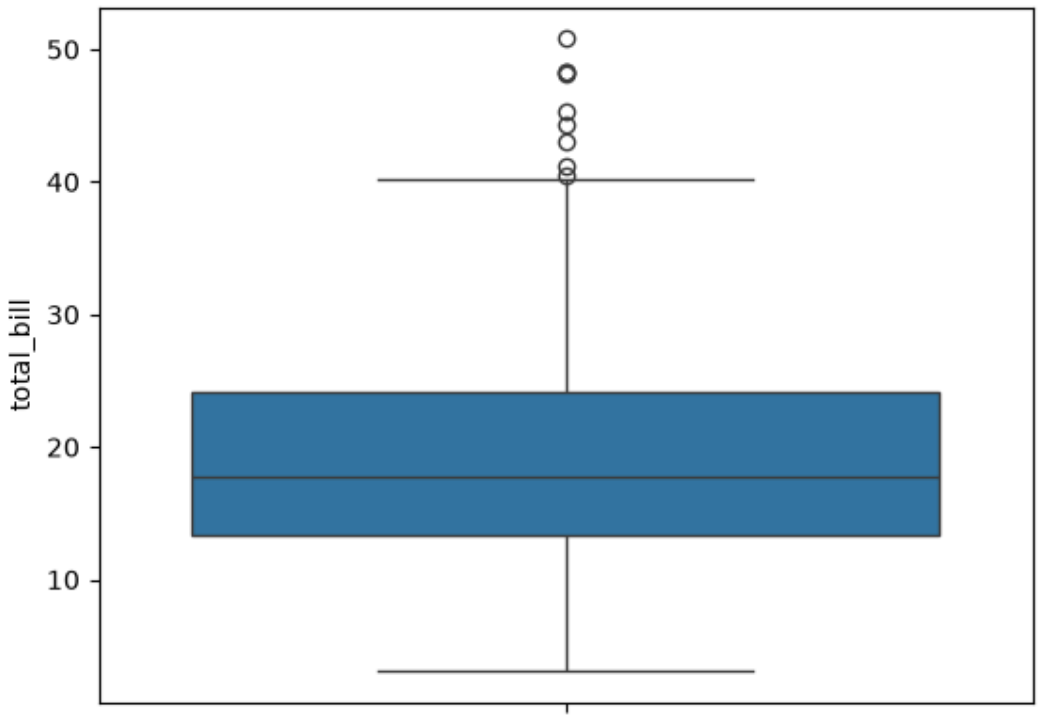
```
              total_bill       tip      size
total_bill      1.000000  0.675734  0.598315
tip             0.675734  1.000000  0.489299
size            0.598315  0.489299  1.000000
```

OUTPUT



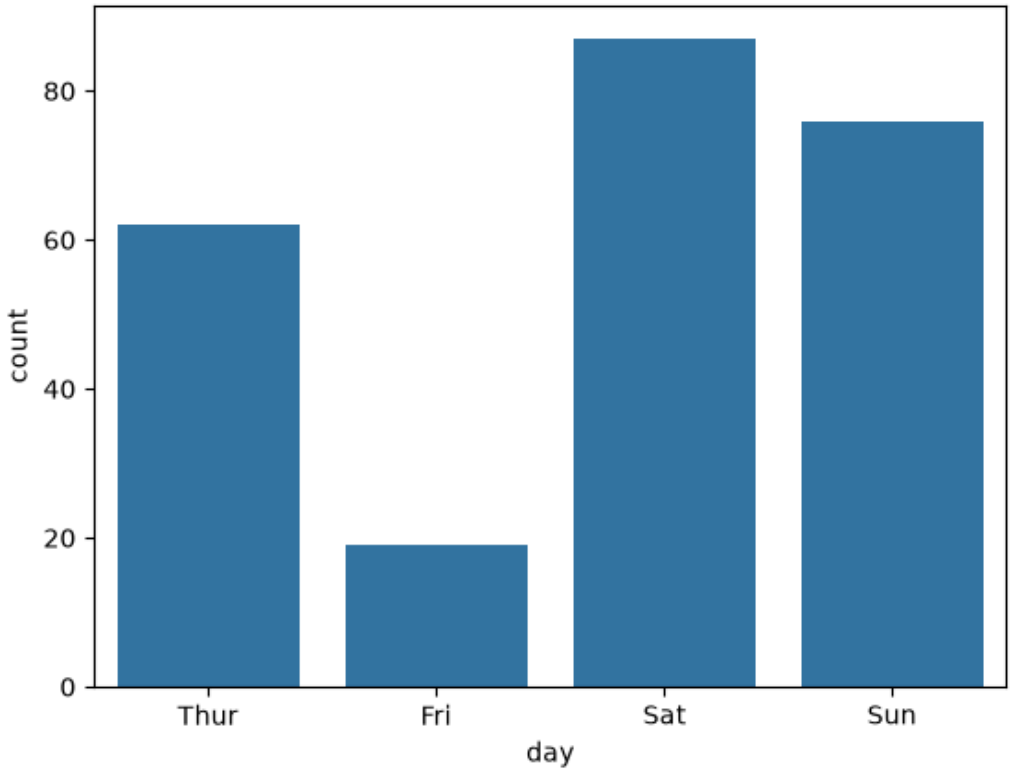
In [7]:
```
sns.boxplot(y=tips["total_bill"])
plt.show()
```

OUTPUT

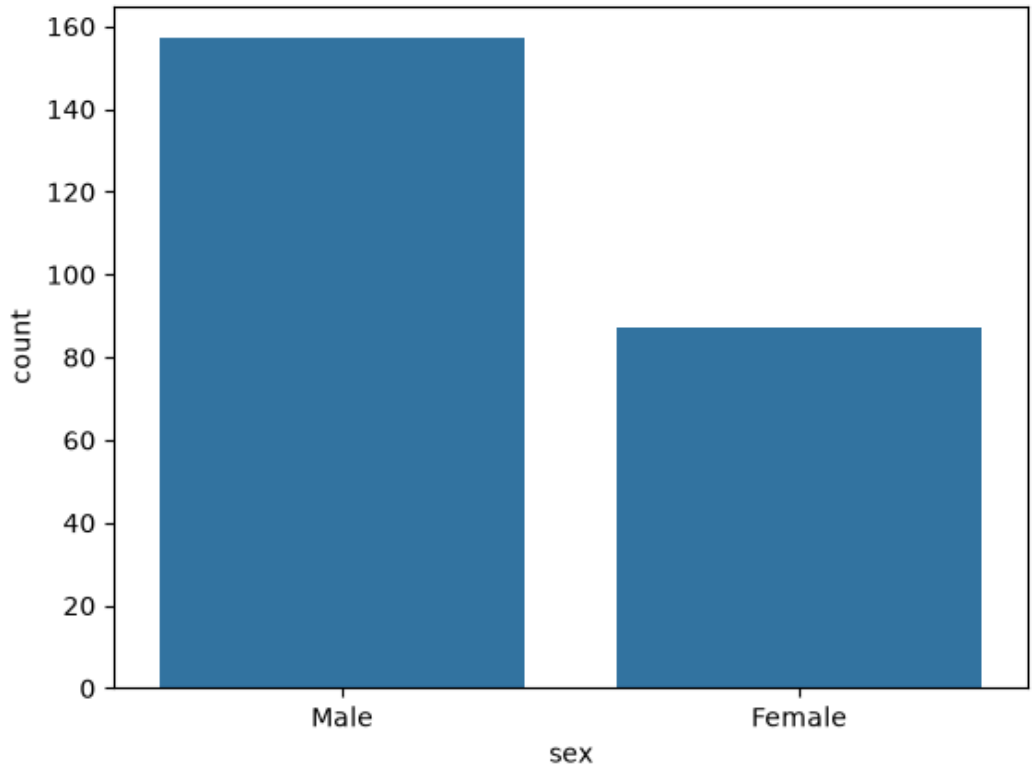


In [8]:
```
sns.countplot(x="day", data=tips)
plt.show()
sns.countplot(x="sex", data=tips)
plt.show()
```

OUTPUT

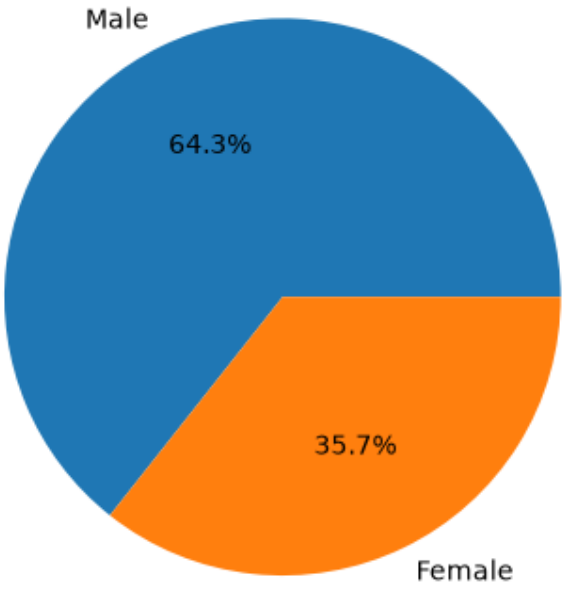


OUTPUT

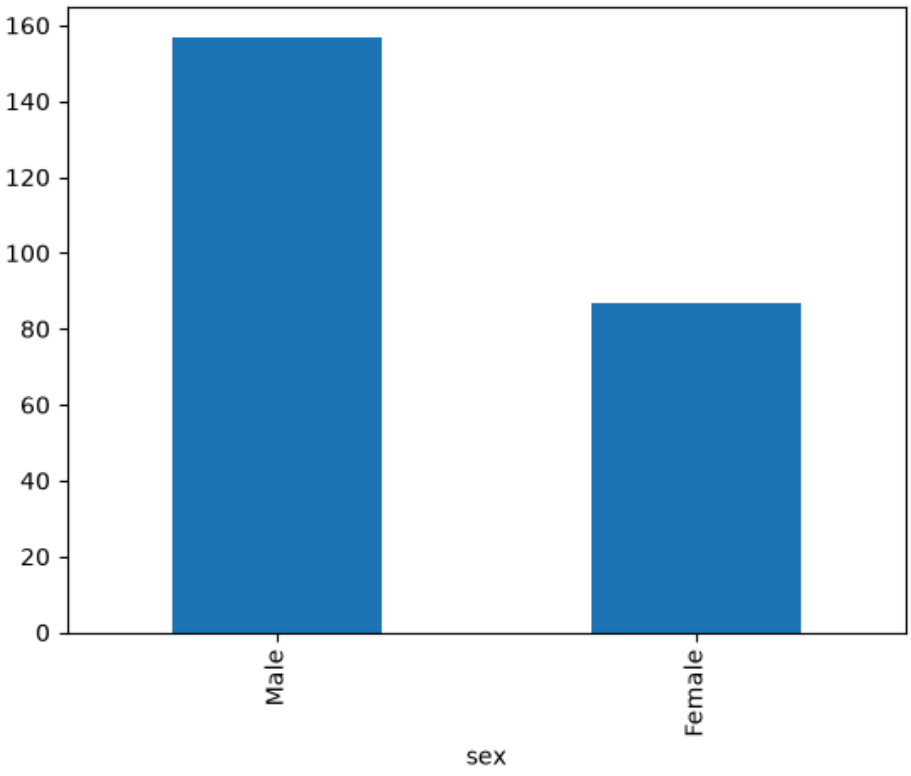


In [9]:
```
tips["sex"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.show()
tips["sex"].value_counts().plot(kind="bar")
plt.show()
```

OUTPUT



OUTPUT



EXPERIMENT 17

EXPERIMENT TO UNDERSTAND LINEAR REGRESSION FOR A GIVEN DATA SET.

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

In [2]:
```
np.random.seed(0)
exp = np.array(
    [
        1.1,
        1.3,
        1.5,
        2.0,
        2.2,
        2.9,
        3.0,
        3.2,
        3.5,
        3.8,
        4.0,
        4.1,
        4.5,
        4.9,
        5.1,
        5.3,
        5.9,
        6.0,
        6.8,
        7.1,
        7.9,
        8.2,
        8.7,
        9.0,
        9.5,
        9.6,
        10.3,
        10.5,
        11.0,
        12.0,
    ]
)
sal = (exp * 9000 + 25000 + np.random.normal(0, 3000, len(exp))).astype(int)
df = pd.DataFrame({"YearsExperience": exp, "Salary": sal})
print(df.info())
print(df.describe())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.830000   78798.166667
std           3.202278   28381.668494
min           1.100000   37900.000000
25%           3.275000   55185.000000
50%           5.200000   72966.000000
75%           8.575000  103019.750000
max          12.000000  137408.000000
```

In [3]:
```
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=0
)
```

In [4]:
```
model = LinearRegression()
model.fit(x_train, y_train)
```

Out[4]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [5]:
```
print("train:", model.score(x_train, y_train))
print("test:", model.score(x_test, y_test))
print("coef:", model.coef_, "intercept:", model.intercept_)
print(model.predict(np.array([[5.0]])))
```

```
train: 0.9854548635386804
test: 0.986084135832942
coef: [[8590.07946135]] intercept: [28307.26566741]
[[71257.66297415]]
```

EXPERIMENT 18

EXPERIMENT TO UNDERSTAND KNN ALGORITHM FOR A GIVEN DATA SET.

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.datasets import load_iris
```

In [2]:
```
iris = load_iris(as_frame=True)
df = iris.frame
df["variety"] = df["target"].map({0: "Setosa", 1: "Versicolor", 2: "Virginica"})
print(df.info())
print(df["variety"].value_counts())
print(df.head())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
 5   variety            150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
None
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target variety
0       0  Setosa
1       0  Setosa
2       0  Setosa
3       0  Setosa
4       0  Setosa
```

In [3]:
```
features = df.iloc[:, 0:4].values
label = df["variety"].values
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=0
)
```

In [4]:
```
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
print(model_KNN.score(xtrain, ytrain))
print(model_KNN.score(xtest, ytest))
```

```
0.95
0.9666666666666667
```

In [5]:
```
print(confusion_matrix(label, model_KNN.predict(features)))
print(classification_report(label, model_KNN.predict(features)))
```

```
[[50  0  0]
 [ 0 45  5]
 [ 0  2 48]]
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.90      0.93        50
   Virginica       0.91      0.96      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

EXPERIMENT 19

EXPERIMENT TO UNDERSTAND LOGISTIC REGRESSION FOR A GIVEN DATA SET.

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
```

In [2]:
```
np.random.seed(0)
n = 400
age = np.random.randint(18, 60, n)
salary = np.random.randint(15000, 150000, n)
purchased = ((age + salary / 5000 + np.random.normal(0, 3, n)) > 45).astype(int)
df = pd.DataFrame({"Age": age, "EstimatedSalary": salary, "Purchased": purchased})
print(df.head())
```

```
   Age  EstimatedSalary  Purchased
0   18            16384          0
1   21            94403          0
2   21            80599          0
3   57           117303          1
4   27            66654          0
```

In [3]:
```
features = df.iloc[:, [0, 1]].values
label = df.iloc[:, 2].values
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=0
)
```

In [4]:
```
finalModel = LogisticRegression(max_iter=1000)
finalModel.fit(x_train, y_train)
print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))
```

```
0.946875
0.95
```

In [5]:
```
print(classification_report(label, finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.91      0.90      0.91       113
           1       0.96      0.97      0.96       287

    accuracy                           0.95       400
   macro avg       0.94      0.93      0.94       400
weighted avg       0.95      0.95      0.95       400
```

EXPERIMENT 20

EXPERIMENT TO UNDERSTAND K-MEANS CLUSTERING ALGORITHM FOR A GIVEN DATA SET.

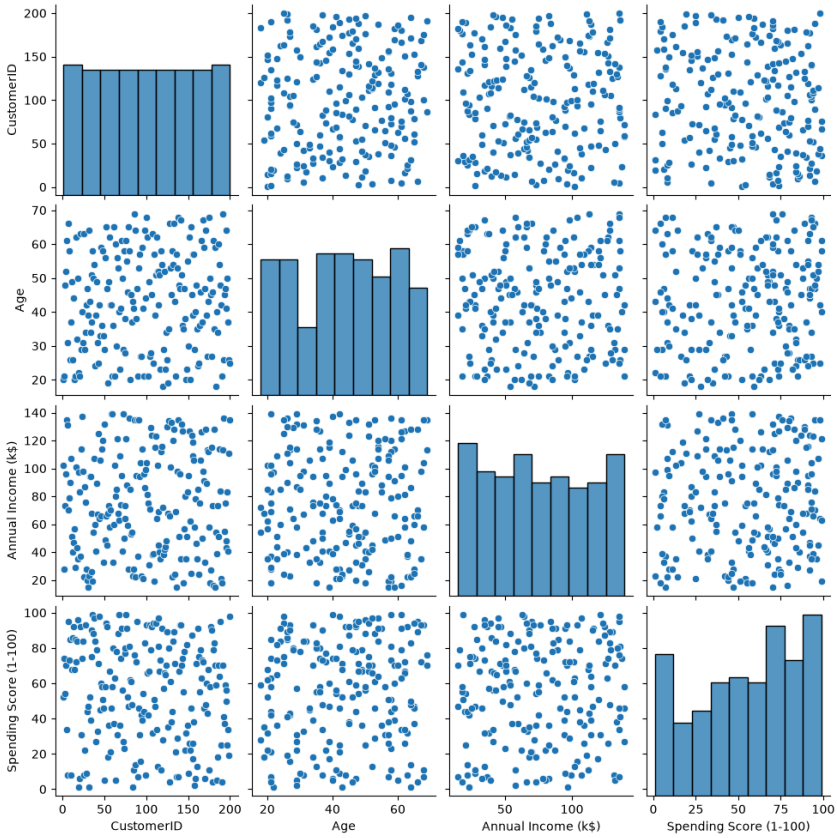
In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
```

In [2]:
```
np.random.seed(0)
n = 200
df = pd.DataFrame(
    {
        "CustomerID": range(1, n + 1),
        "Gender": np.random.choice(["Male", "Female"], n),
        "Age": np.random.randint(18, 70, n),
        "Annual Income (k$)": np.random.randint(15, 140, n),
        "Spending Score (1-100)": np.random.randint(1, 100, n),
    }
)
print(df.info())
print(df.head())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int32
 3   Annual Income (k$)      200 non-null    int32
 4   Spending Score (1-100)  200 non-null    int32
dtypes: int32(3), int64(1), str(1)
memory usage: 5.6 KB
None
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   20                 102                      52
1           2  Female   21                  28                      74
2           3  Female   48                  73                      54
3           4    Male   52                  96                      70
4           5  Female   61                 135                      34
```

In [3]:
```
sns.pairplot(df)
plt.show()
```

OUTPUT

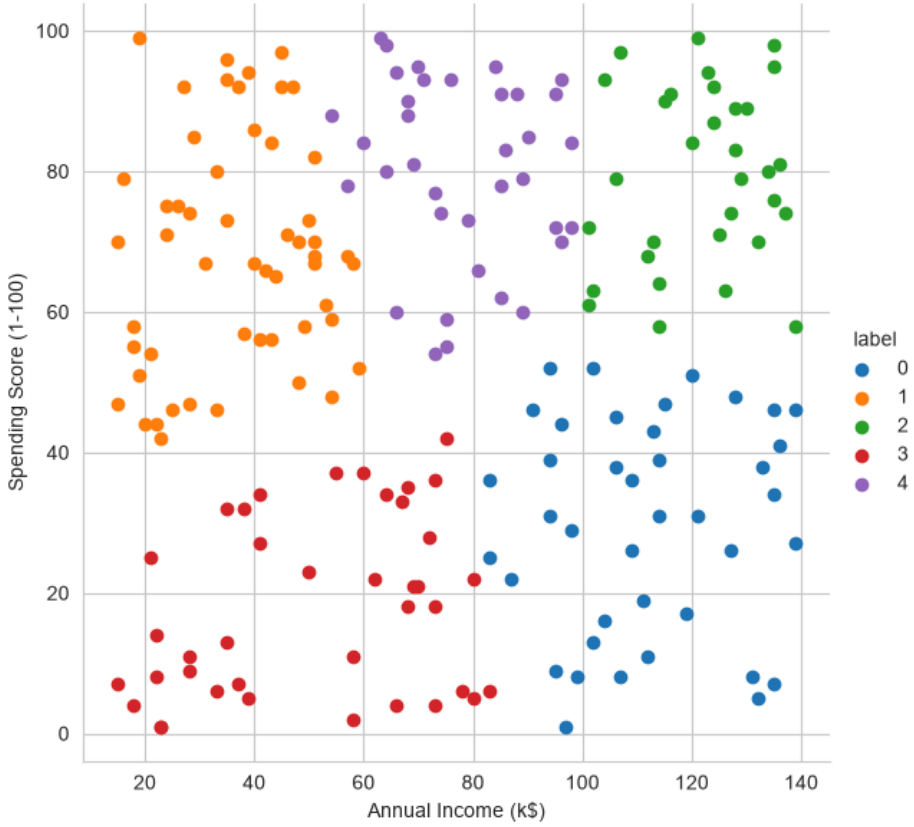


In [4]:
```
features = df.iloc[:, [3, 4]].values
model = KMeans(n_clusters=5, n_init=10, random_state=0)
model.fit(features)
Final = df.iloc[:, [3, 4]].copy()
Final["label"] = model.predict(features)
print(Final.head())
```

```
   Annual Income (k$)  Spending Score (1-100)  label
0                 102                      52      0
1                  28                      74      1
2                  73                      54      4
3                  96                      70      4
4                 135                      34      0
```

In [5]:
```
sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=6).map(
    plt.scatter, "Annual Income (k$)", "Spending Score (1-100)"
).add_legend()
plt.show()
```

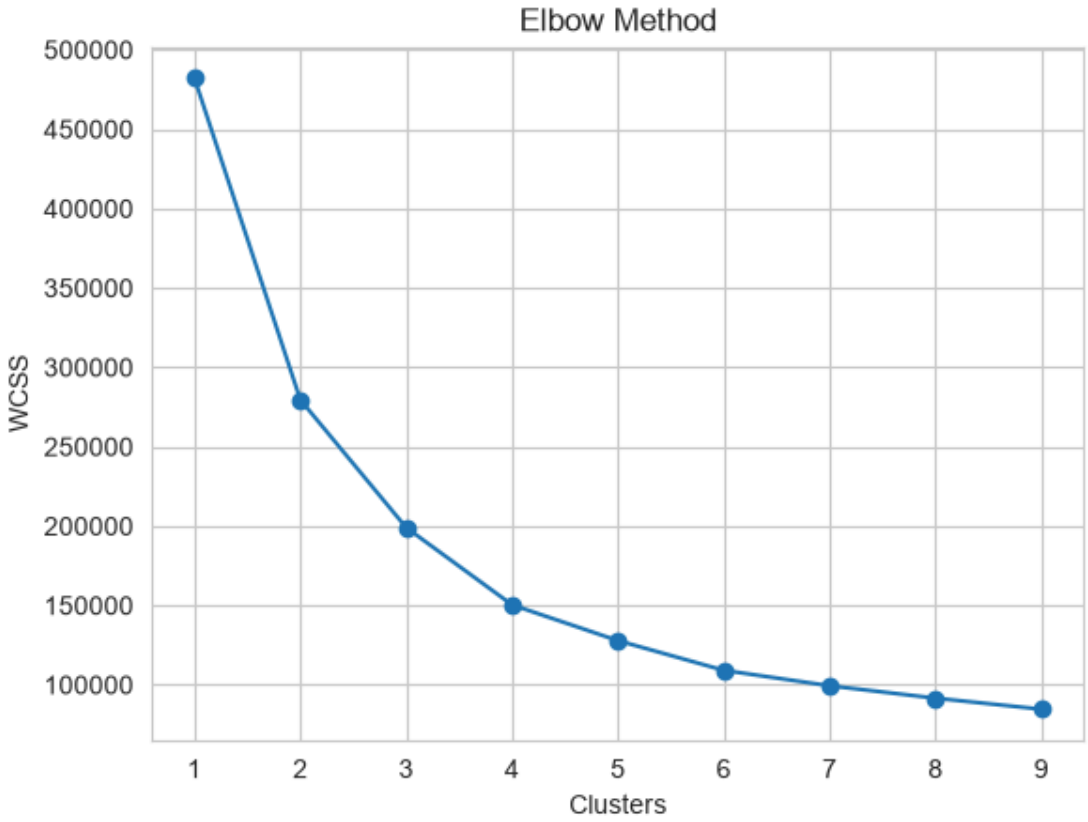
OUTPUT



In [6]:

```
features_el = df.iloc[:, [2, 3, 4]].values
wcss = []
for i in range(1, 10):
    m = KMeans(n_clusters=i, n_init=10, random_state=0)
    m.fit(features_el)
    wcss.append(m.inertia_)
plt.plot(range(1, 10), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Clusters")
plt.ylabel("WCSS")
plt.show()
```

OUTPUT


Elbow Method
WCSS
500000
450000
400000
350000
300000
250000
200000
150000
100000
1
2
3
4
5
6
7
8
9
Clusters